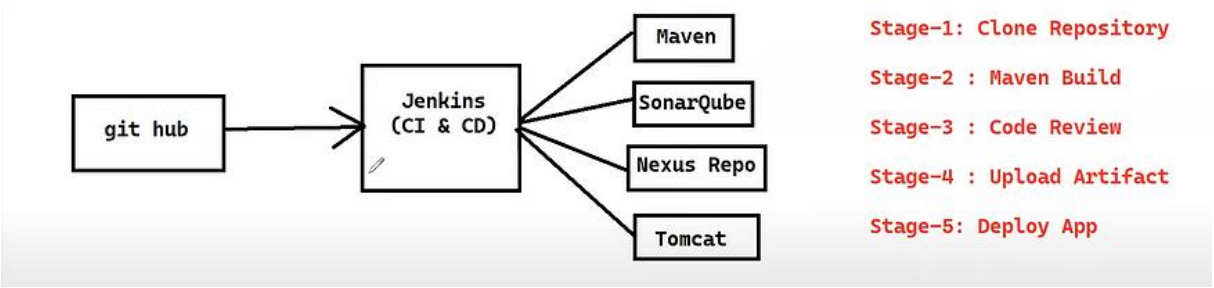
Configuration du pipeline DevOps CICD via { Jenkins | Maven | Sonarqube | Nexus | Tomcat}

Donc dans le cadre de cette configuration, nous allons d'abord utiliser plusieurs outils, notre projet est disponible sur GitHub où le code source du projet sera disponible. Nous prendrons donc un projet de GitHub et le déploierons en différentes étapes à l'aide de la structure CICD.



Afin de créer et de déployer cette application, nous utilisons **Jenkins**. Chez Jenkins, nous utilisons le concept de pipeline CICD. Ce logiciel Jenkins va donc cloner le dépôt depuis GitHub et communiquera avec **Maven.**

Maven : Il s'agit d'un outil de build utilisé pour effectuer le processus de build des applications Java.

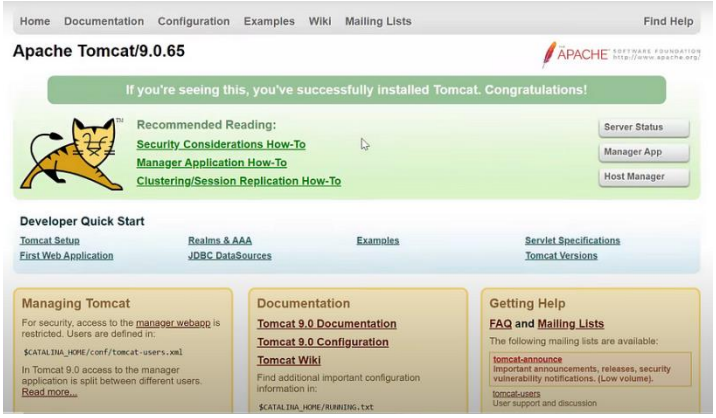
Maintenant, Jenkins va cloner le référentiel depuis le github et maven va compiler et empaqueter notre application Java. Une fois le processus de construction terminé, Jenkins devrait pouvoir communiquer avec le Sonarqube.

SonarQube : Pour effectuer la révision du code, Sonarqube est un logiciel de vérification de la qualité du code dans le pipeline.

Après avoir effectué la révision du code de notre application, nous souhaitons créer un artefact dans le référentiel Nexus.

Nexus : Nexus est le référentiel où sont placés tous les artefacts.

Après avoir téléchargé l'artefact dans le référentiel Nexus, nous souhaitons déployer le fichier war de l'application sur le serveur Apache Tomcat.

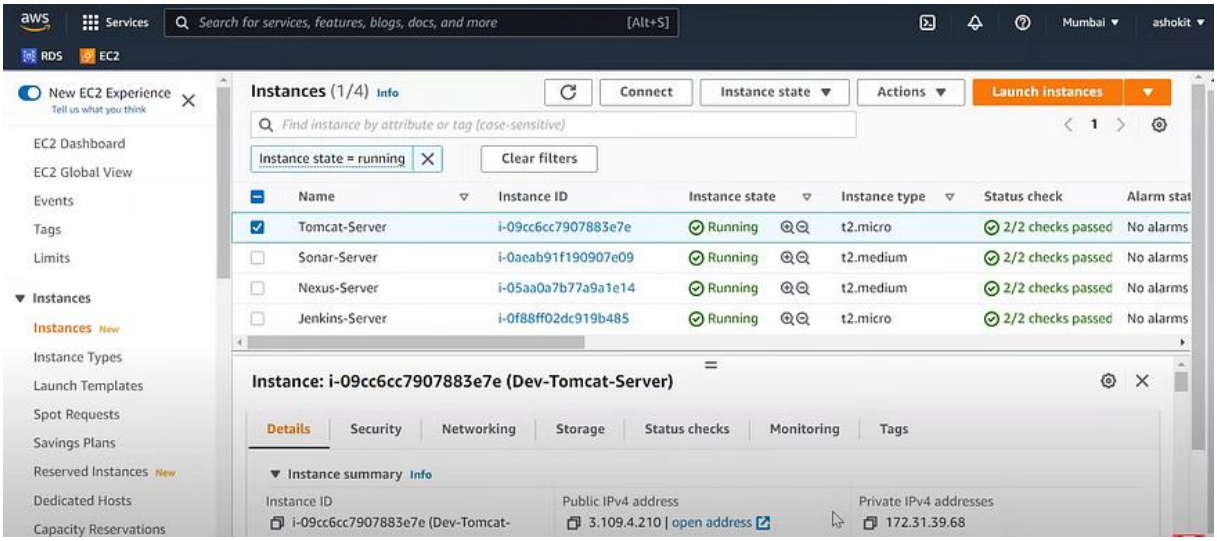


Serveur Apache Tomcat : est un conteneur utilisé pour exécuter nos applications Web.

Nous utiliserons le serveur Tomcat pour exécuter notre application Java. Jenkins est chargé de communiquer avec tous les outils pour automatiser le processus de création et de déploiement d'applications. Afin d'effectuer toutes ces étapes, nous allons créer un pipeline dans Jenkins.

Après cela, nous créerons AWS Linux Machine, vous pourrez utiliser n'importe quelle machine dans votre zone de confort.

Nous allons créer 4 serveurs comme décrit ci-dessous :



Pour ce projet, nous utiliserons Linux Centos pour l'installation du serveur Tomcat.

Pour l'installation du serveur Tomcat

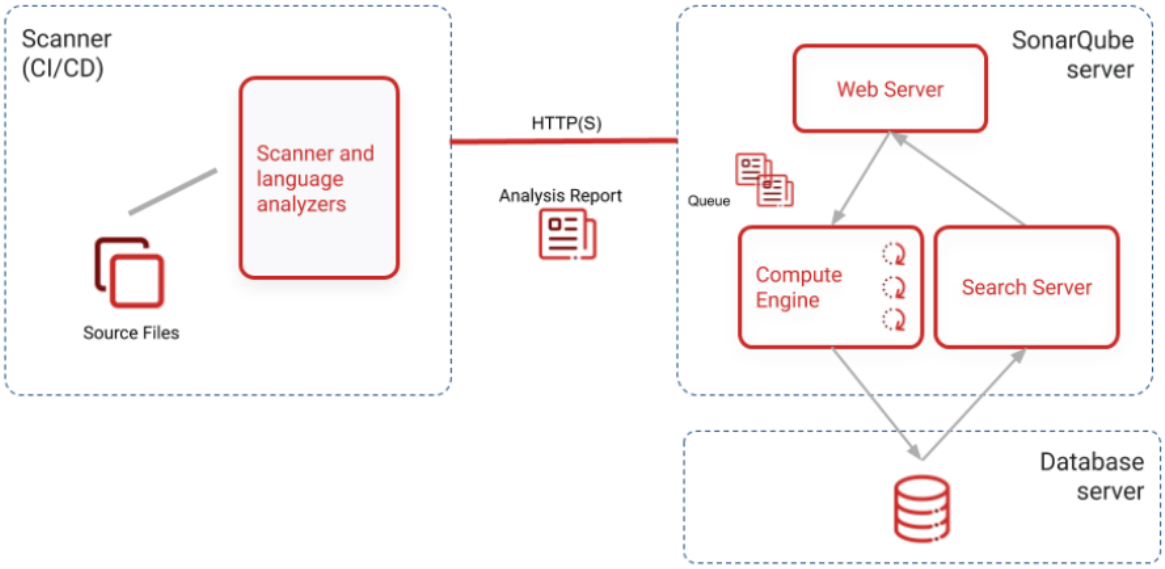
Pour l'installation de SonarQube

SonarQube est un outil d'inspection continue qui peut être utilisé pour tester la qualité du code. Il analyse le code source et nous envoie le rapport analytique pour vérifier la qualité finale.

Il est disponible en tant que plate-forme open source et prend en charge plusieurs langages de programmation tels que Java, Python , Javascript, TypeScript, COBOL, HTML, XML, C#, C/C++ Apex, Object-C, Swift, Kotlin , Ruby, Scala, CSS, ABAP, etc. Cependant, certaines langues nécessitent une licence commerciale pour fonctionner.

Vous n'avez pas besoin de modifier votre flux de travail ou d'apprendre de nouveaux outils pour installer SonarQube, car il peut facilement être intégré à des outils de construction courants tels que Ant, Maven, Make, Gradle, MS Build, etc. Tout ce que vous avez à faire est d'ajouter les plugins appropriés pour une analyse plus fluide dans votre processus de construction.

Une instance unique de SonarQube comprend trois composants :



1. Le serveur SonarQube exécute les processus suivants :
   - Un serveur Web qui dessert l'interface utilisateur de SonarQube.
   - Un serveur de recherche basé sur Elasticsearch.
   - Le moteur de calcul chargé de traiter les rapports d'analyse de code et de les enregistrer dans la base de données SonarQube.
2. La base de données pour stocker les éléments suivants :
   - Métriques et problèmes de qualité et de sécurité du code générés lors des analyses de code.
   - La configuration de l'instance SonarQube.
3. Un ou plusieurs scanners exécutés sur vos serveurs de build ou d'intégration continue pour analyser les projets.

Suivez ces étapes pour l'installer via docker compose :

a) Créer le fichier docker-compose.yml suivant dans un répertoire dédié

```
version: "3"

services:
  sonarqube:
    image: sonarqube:community
    depends_on:
      - db
    environment:
      SONAR_JDBC_URL: jdbc:postgresql://db:5432/sonar
      SONAR_JDBC_USERNAME: sonar
      SONAR_JDBC_PASSWORD: password
    volumes:
      - sonarqube_data:/opt/sonarqube/data
      - sonarqube_extensions:/opt/sonarqube/extensions
      - sonarqube_logs:/opt/sonarqube/logs
    ports:
      - "9000:9000"
    restart: always
  db:
    image: postgres:12
    environment:
      POSTGRES_USER: sonar
      POSTGRES_PASSWORD: password
    volumes:
      - postgresql:/var/lib/postgresql
      - postgresql_data:/var/lib/postgresql/data
    restart: always


volumes:
  sonarqube_data:
  sonarqube_extensions:
  sonarqube_logs:
  postgresql:
  postgresql_data:
```

b) Il faut configurer Linux avant de lancer le template en appliquant le paramètre suivant

```
root# sysctl -w vm.max_map_count=262144
root# echo "vm.max_map_count=262144" > /etc/sysctl.d/98-vm.max_map_count.conf
```

c) Exécutez maintenant le fichier de composition à l'aide de la commande Docker compose :

```
$ docker compose up -d
```

d) Répertoriez les conteneurs en cours d'exécution pour voir si les conteneurs SonarQube et postgreSQL sont présents.

```
$ docker compose ps
```

e) Ouvrez maintenant dans votre navigateur http://localhost:9000 et connectez-vous à votre compte par défaut admin dont le mot de passe est admin.

Pour une installation rapide de Nexus

a) Tout d'abord, nous commencerons par créer un volume pour conserver les données persistantes au cas où le conteneur s'arrêterait pour une raison quelconque.

```
$ docker volume create --name nexus-data
```

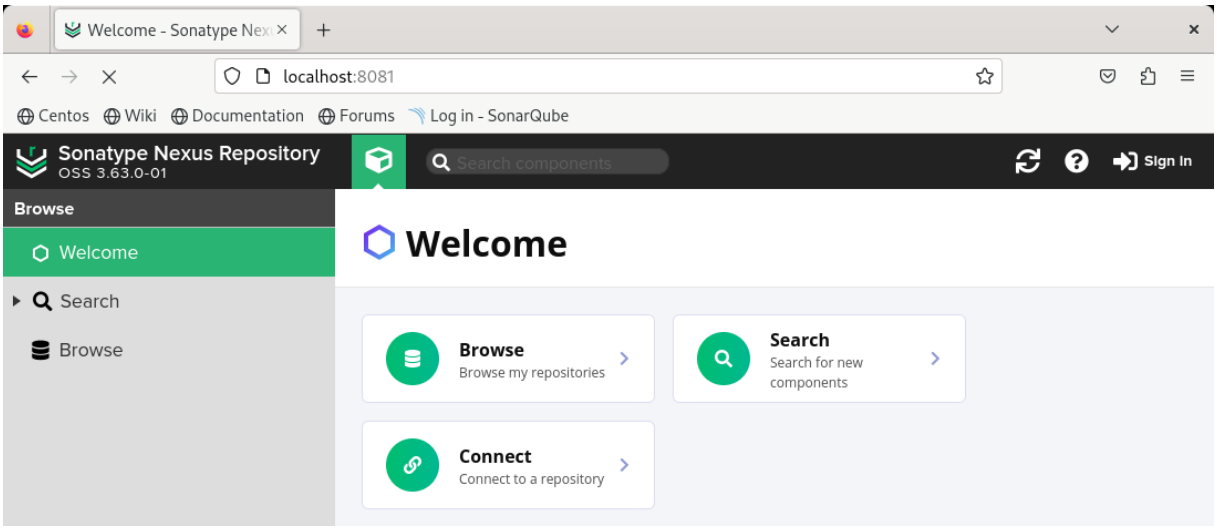
b) Ensuite, nous allons exécuter le conteneur avec l'image sonatype/nexus3

```
$ docker run -d -p 8081:8081 --name nexus -v nexus-data:/nexus-data
sonatype/nexus3
```

Par défaut, le gestionnaire de référentiel Nexus s'exécute sur le port 8081

Maintenant, nous allons lui donner une minute ou deux pour s'initialiser

Maintenant, en visitant l'url http://localhost:8081, nous pouvons voir que Nexus s'est initialisé



c) La prochaine étape que nous devons faire est d'obtenir le mot de passe administrateur qui est généré automatiquement lors du lancement initial du conteneur.

Comme indiqué dans la documentation, l'utilisateur par défaut est admin et le mot de passe généré de manière unique se trouve dans le fichier admin.password à l'intérieur du volume.

Obtenir le mot de passe administrateur

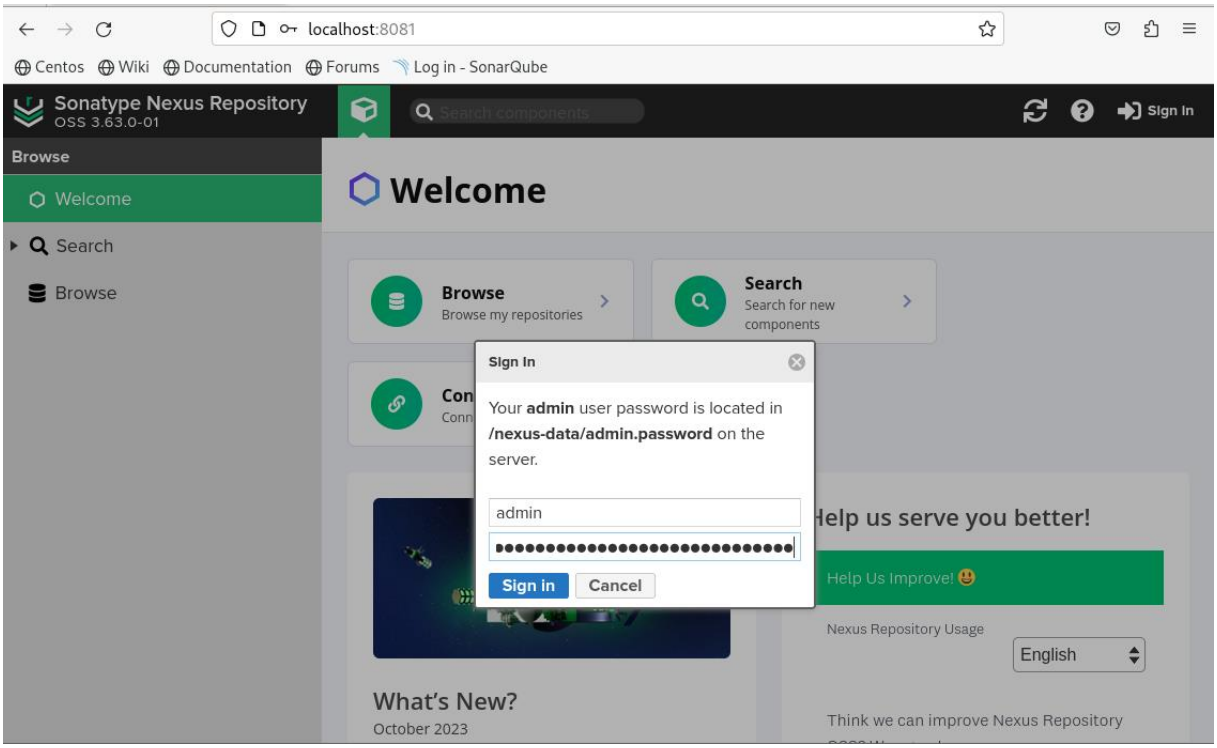
Nous transmettrons le mot de passe à notre console en utilisant docker exe

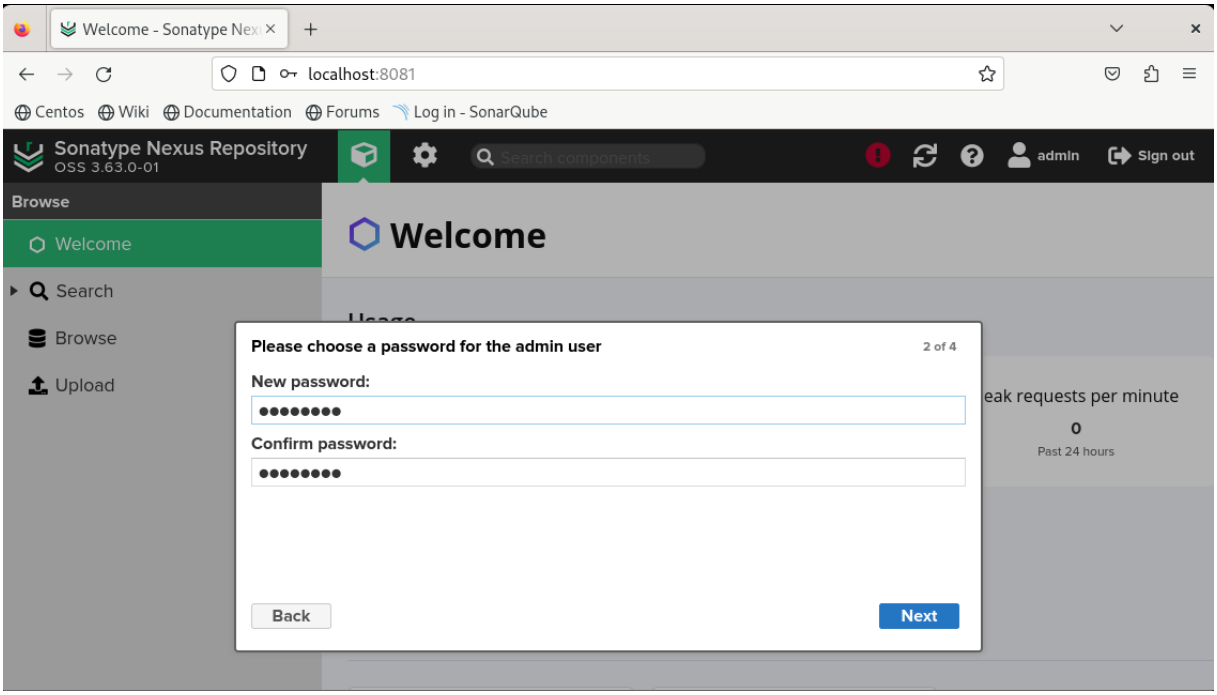
```
$ docker container exec nexus cat /nexus-data/admin.password
```

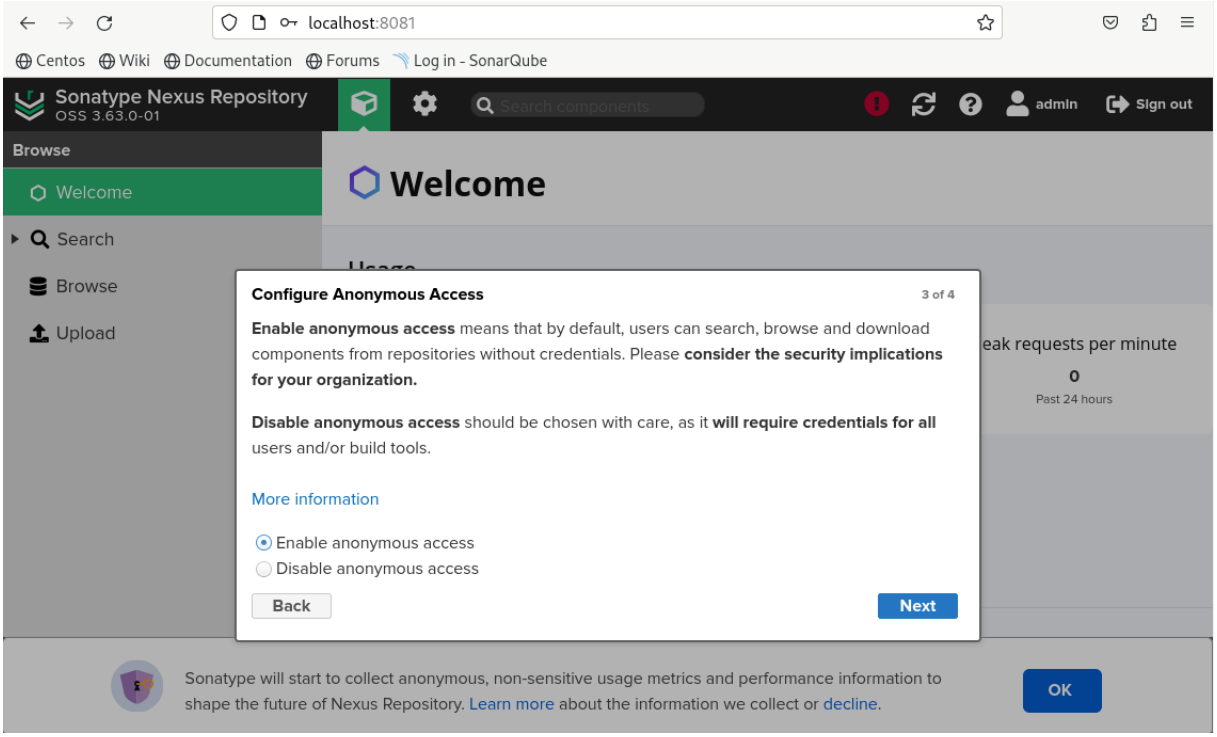
ce qui devrait renvoyer quelque chose comme ce qui suit

```
[admuser@server sonarqube]$ docker container exec nexus cat /nexus-data/admin.password
40cbba1e-e29e-42c7-9cb6-e589b059e4c8[admuser@server sonarqube]$
```

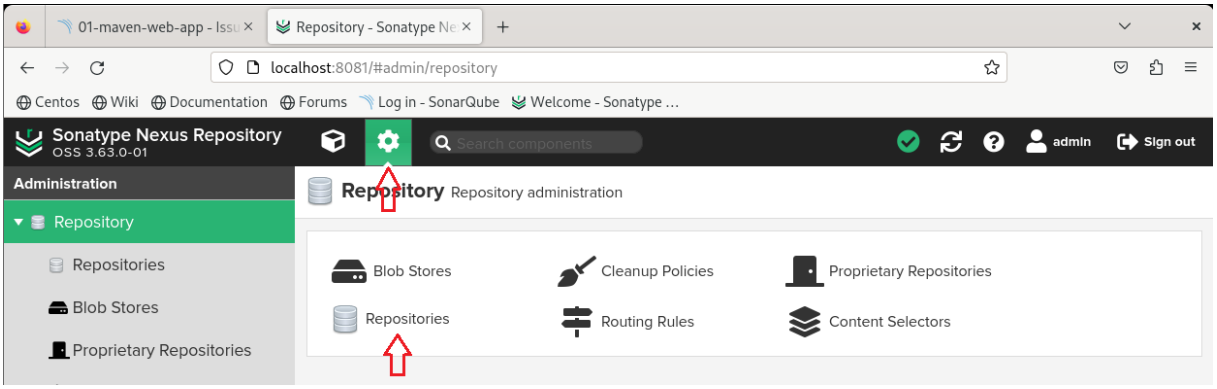
Nous pouvons maintenant nous connecter au panneau d'administration de Nexus


localhost:8081
Centos
Wiki
Documentation
Forums
Log in - SonarQube
Sonatype Nexus Repository
OSS 3.63.0-01
Sign In
Browse
Welcome
Search
Browse
Welcome
Browse
Browse my repositories
Search
Search for new components
Sign In
Your admin user password is located in /nexus-data/admin.password on the server.
admin
Sign in
Cancel
Help us serve you better!
Help Us Improve!
Nexus Repository Usage
English
What's New?
October 2023
Think we can improve Nexus Repository

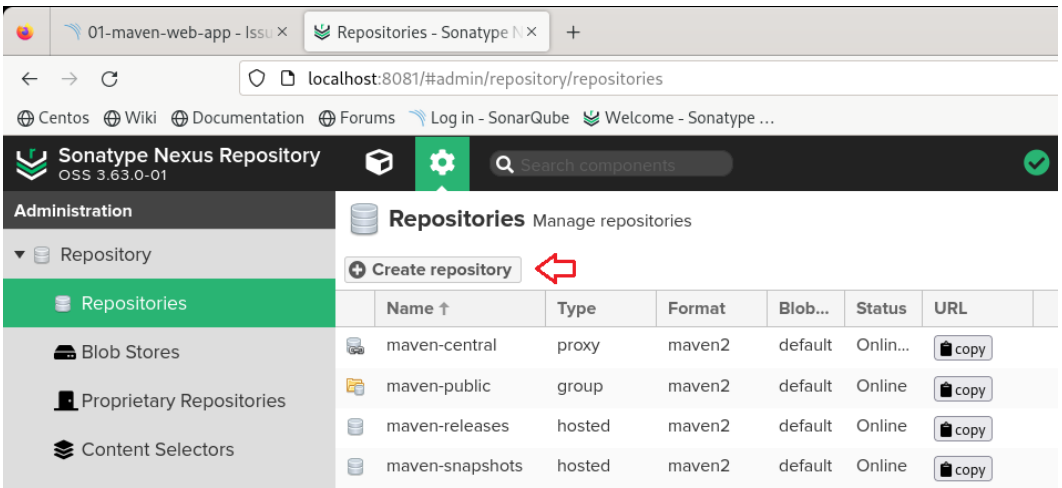

Welcome - Sonatype Nex
localhost:8081
Centos
Wiki
Documentation
Forums
Log in - SonarQube
Sonatype Nexus Repository
OSS 3.63.0-01
admin
Sign out
Browse
Welcome
Search
Browse
Upload
Welcome
Please choose a password for the admin user
2 of 4
New password:
Confirm password:
Back
Next
0
Past 24 hours

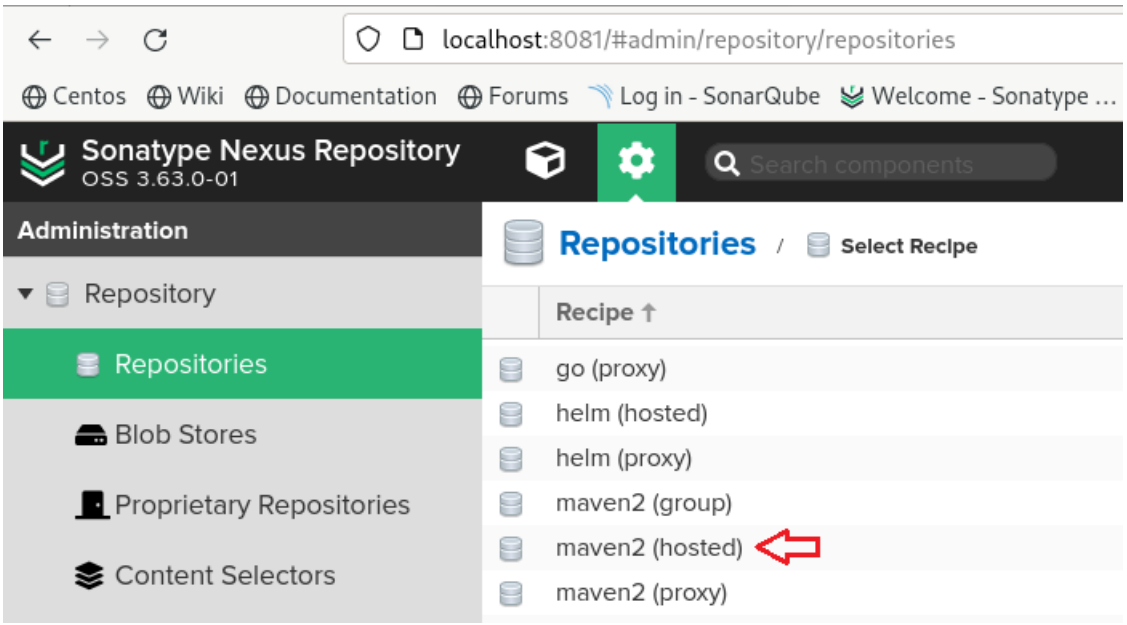


et à partir de là, nous pouvons configurer Nexus et créer différents référentiels en fonction des besoins de notre application.

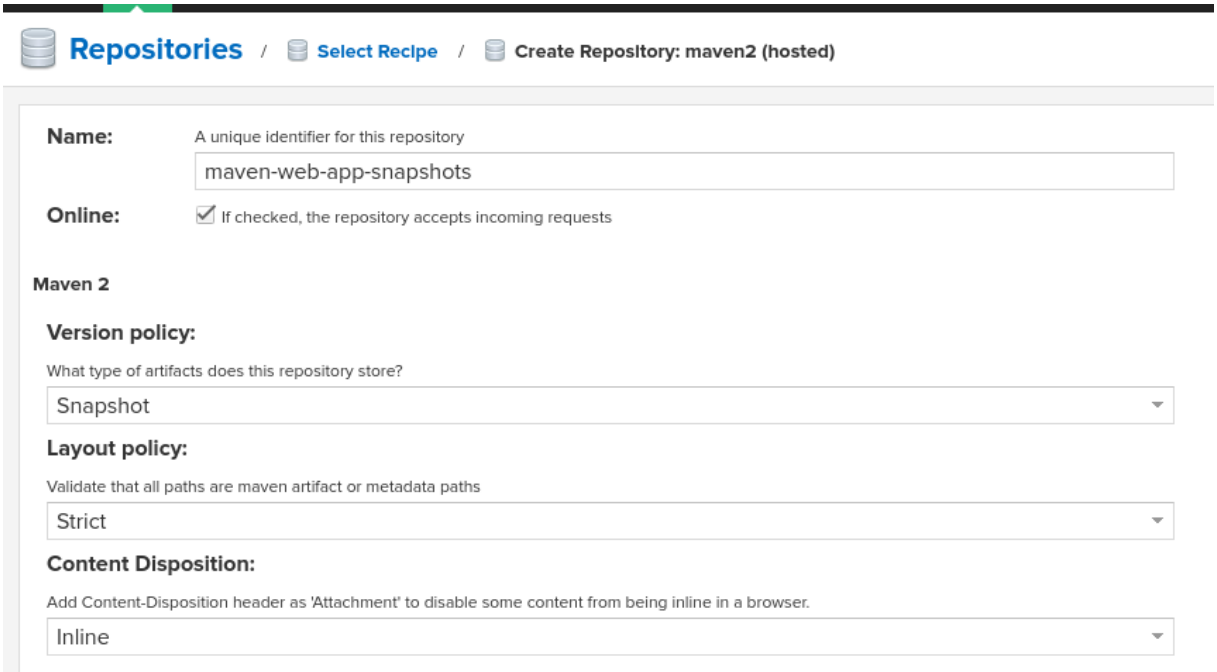


-

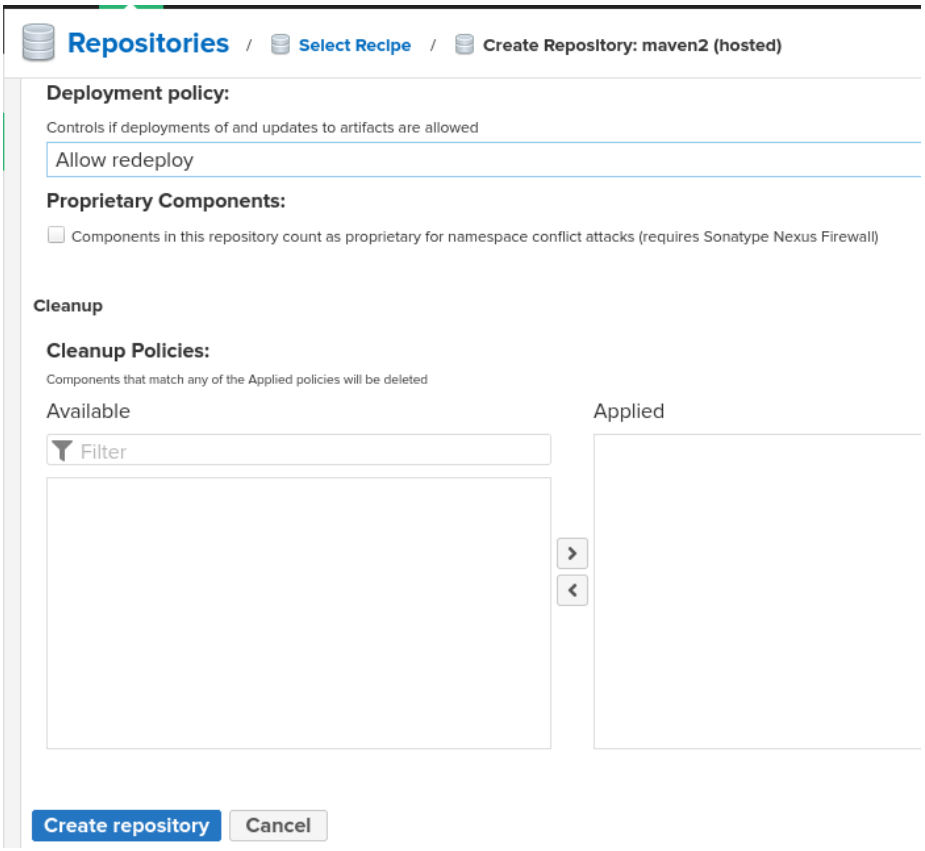



localhost:8081/#admin/repository/repositories
Centos
Wiki
Documentation
Forums
Log in - SonarQube
Welcome - Sonatype ...
Sonatype Nexus Repository
OSS 3.63.0-01
Search components
Administration
Repository
Repositories
Blob Stores
Proprietary Repositories
Content Selectors
Repositories / Select Recipe
Recipe
go (proxy)
helm (hosted)
helm (proxy)
maven2 (group)
maven2 (hosted)
maven2 (proxy)

-


Repositories / Select Recipe / Create Repository: maven2 (hosted)
Name:
A unique identifier for this repository
maven-web-app-snapshots
Online:
If checked, the repository accepts incoming requests
Maven 2
Version policy:
What type of artifacts does this repository store?
Snapshot
Layout policy:
Validate that all paths are maven artifact or metadata paths
Strict
Content Disposition:
Add Content-Disposition header as 'Attachment' to disable some content from being inline in a browser.
Inline

-

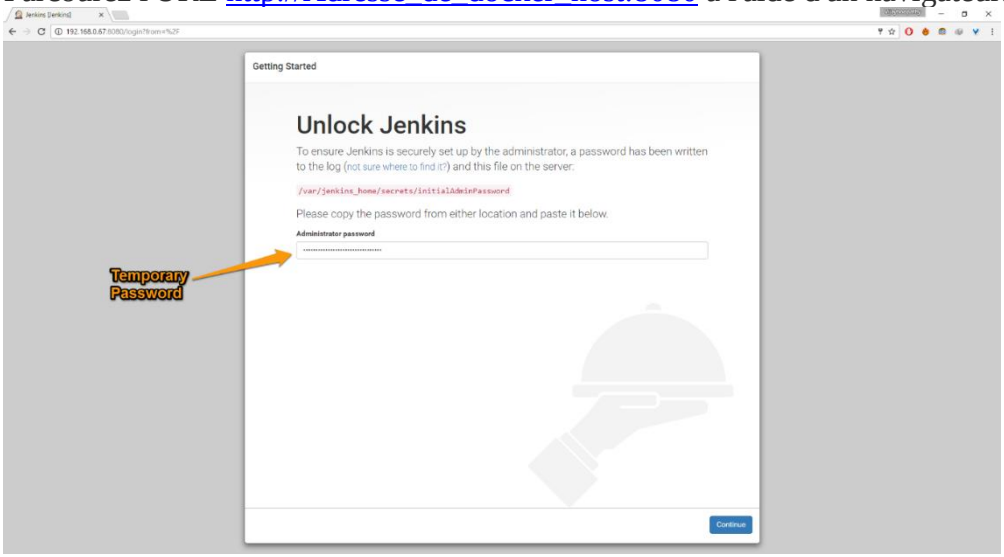

Repositories / Select Recipe / Create Repository: maven2 (hosted)
Deployment policy:
Controls if deployments of and updates to artifacts are allowed
Allow redeploy
Proprietary Components:
Components in this repository count as proprietary for namespace conflict attacks (requires Sonatype Nexus Firewall)
Cleanup
Cleanup Policies:
Components that match any of the Applied policies will be deleted
Available
Applied
Filter
Create repository
Cancel

Pour l'installation de Jenkins

```
$ sudo -i
# wget -O /etc/yum.repos.d/jenkins.repo
https://pkg.jenkins.io/redhat-stable/jenkins.repo
# rpm --import https://pkg.jenkins.io/redhat-
stable/jenkins.io.key
# yum upgrade
# yum install java-11-openjdk*
# cat > /etc/profile.d/java.sh <<'EOF'
export JAVA_HOME=$(dirname $(dirname $(readlink $(readlink
$(which java)))))
export PATH=$PATH:$JAVA_HOME/bin
EOF
# source /etc/profile.d/java.sh
# yum install jenkins
# systemctl enable --now jenkins
# hostnamectl set-hostname jenkins-server.lab.local
# IP=$(hostname -I | awk '{print $1}') eval 'echo "$IP
jenkins-server  jenkins-server.lab.local" >> /etc/hosts'
```

Accès à l'interface utilisateur Web de Jenkins:

Parcourez l'URL http://Adresse_de_docker_host:8080 à l'aide d'un navigateur.



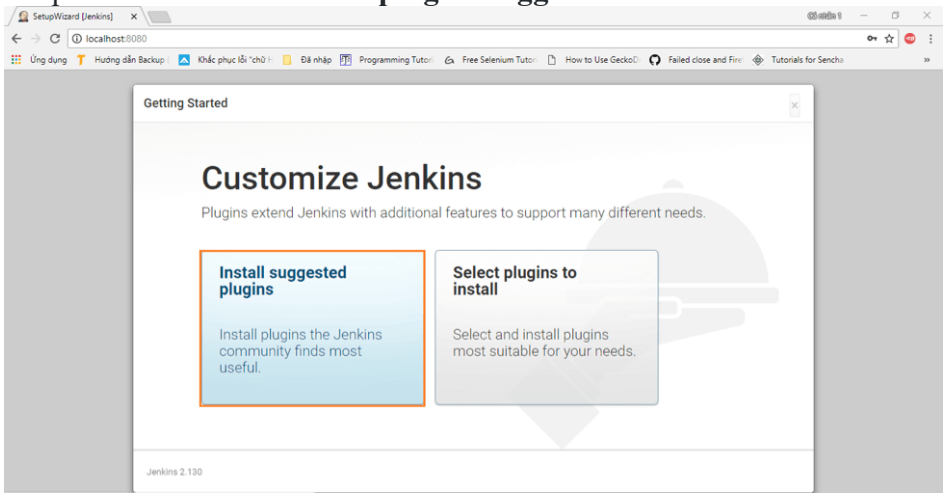
- Puisque, nous accédons à l'interface Web de Jenkins pour la première fois, nous avons besoin du mot de passe administrateur pour nous y connecter.

Le chemin d'accès au fichier de mot de passe a été fourni par l'interface Web Jenkins.

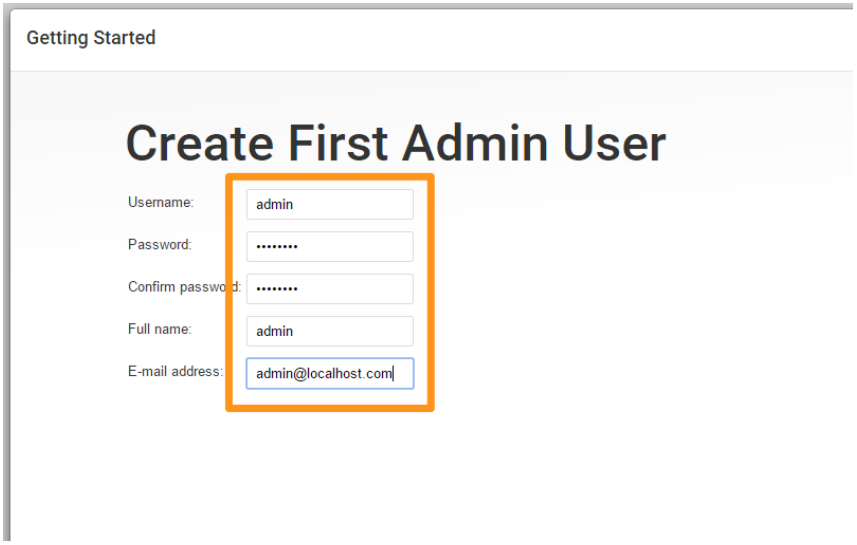
```
# cat /var/lib/jenkins/secrets/initialAdminPassword
```

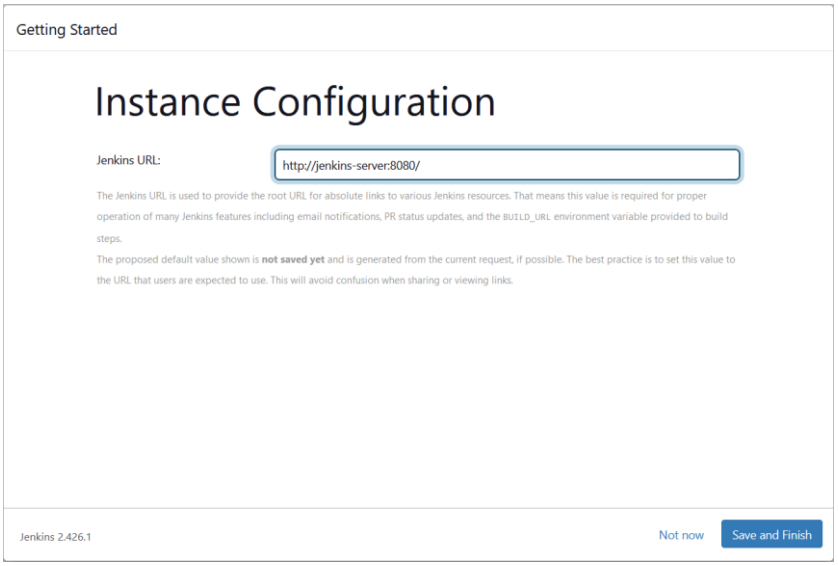
Connectez-vous à l'interface Web de Jenkins en utilisant ce mot de passe.

- Cliquez sur **Sélectionner les plugins suggérés**.



Créer un utilisateur administrateur



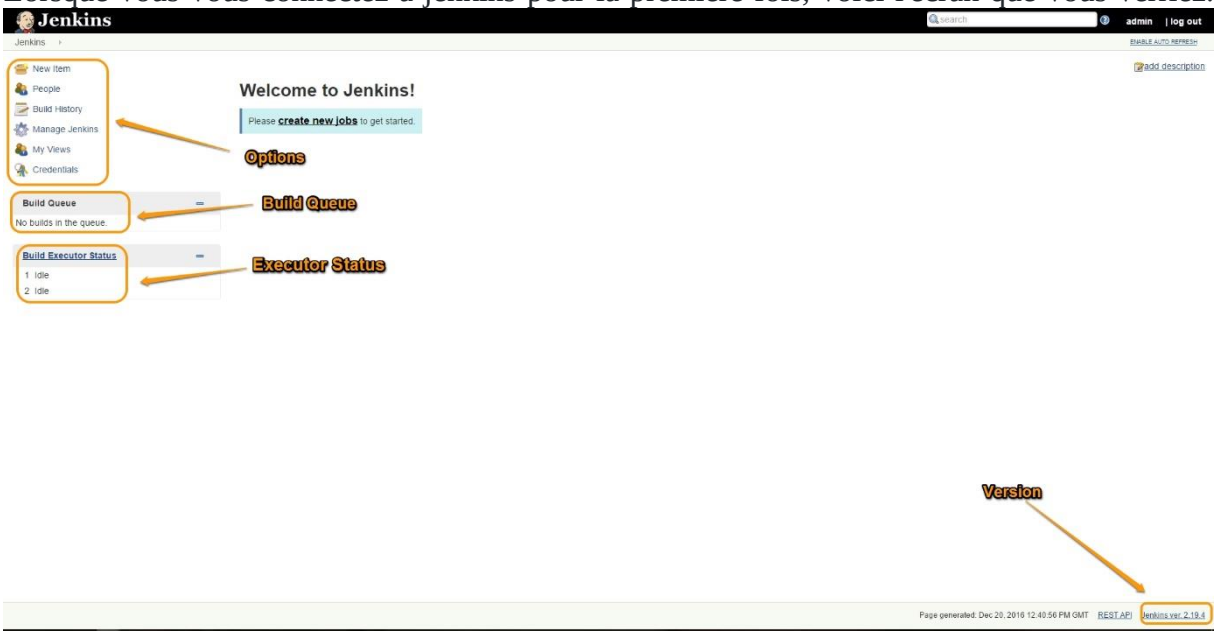


Maintenant, nous avons installé Jenkins avec succès et nous pouvons procéder aux configurations

Configurations Jenkins

Se familiariser avec la console Jenkins

Lorsque vous vous connectez à jenkins pour la première fois, voici l'écran que vous verriez.



- Sur le côté gauche de l'écran, en haut se trouve le menu pour créer de nouveaux projets, pour gérer jenkins, pour créer des utilisateurs, etc.
- Juste en dessous du menu se trouve la file d'attente de construction. Tous les travaux programmés pour s'exécuter sont ajoutés à la file d'attente et apparaîtront ici.
- Sous la file d'attente de construction se trouve le statut de l'exécuteur de compilation. Cela montre l'état des travaux en cours d'exécution en temps réel.
- En bas à droite de la page se trouvent les informations sur la version de jenkins affichées.

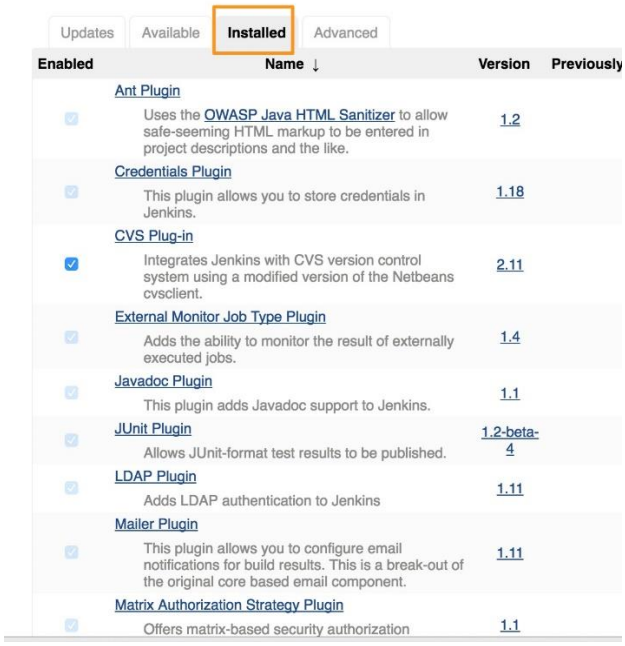
Plugins

La vraie force de Jenkins réside dans son écosystème riche en plugins. C'est ainsi que les outils s'intègrent à jenkins pour créer un flux de travail CI. Vous voulez déclencher des jobs jenkins après chaque changement dans git, vous avez un plugin pour cela. Vous souhaitez envoyer une notification à vos développeurs sur une compilation réussie ou échouée, vous disposez d'un plugin de notification. Vous souhaitez utiliser un outil pour récupérer ou pousser les artefacts de construction, vous avez un plugin pour cela. C'est ainsi que la plupart des outils parlent à jenkins.

Dans ce Lab, nous allons voir un processus simple pour installer des plugins. Dans ce cadre, nous allons installer un plugin qui nous aidera à intégrer jenkins à notre référentiel git.

Explorer les configurations de plugins

- Dans «Administrer Jenkins», sélectionnez l'option «Gestion des plugins».
- Dans le volet Gérer les plugins, vous verrez les onglets suivants,
    - Mises à jour
    - Disponibles
    - Installés

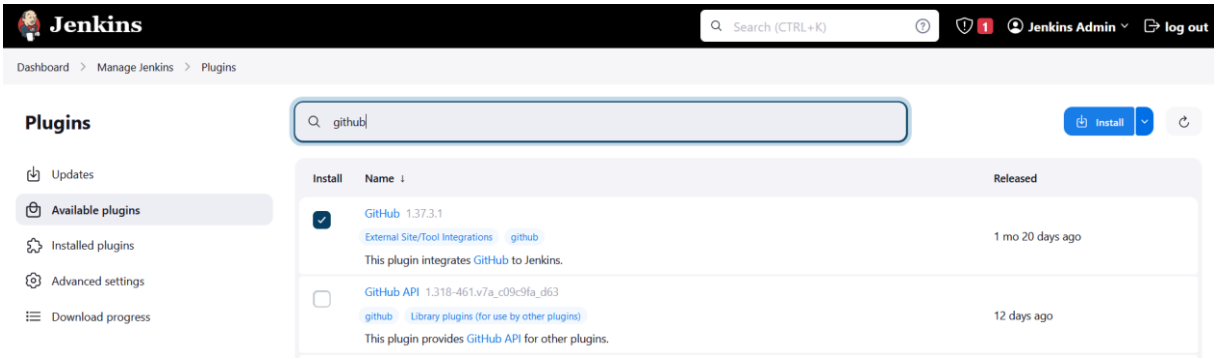


- Avancé

Sélectionnez "Installés" pour afficher la liste des plugins préinstallés avec jenkins.

Installer le plugin Github

- Dans "Gérer les plugins", sélectionnez l'onglet **Disponibles**.
- Dans le coin supérieur droit, vous devriez voir une zone de filtre, commencez à taper le terme de recherche dans cette zone. Pour cet exemple, nous **taperions github**.

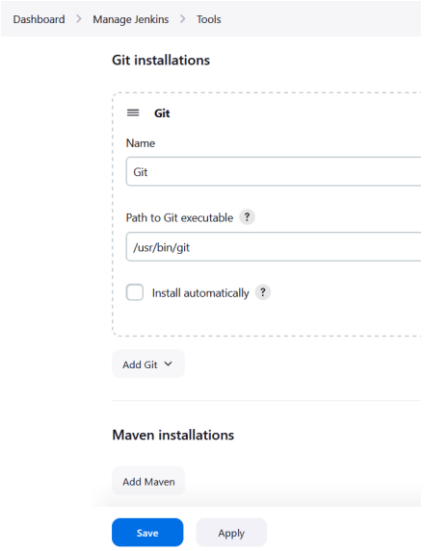


Installer Git sur le serveur jenkins

```
# yum install git -y
# which git
/usr/bin/git
```

Une fois installé il faut configurer le plugin :

Dashboard -> Tools



Apply & Save

Configurer le Global Tool JDK

Sur le serveur jenkins :

```
# echo $JAVA_HOME
/usr/lib/jvm/java-17-openjdk-17.0.6.0.10-3.el9.x86_64
```

Apply & Save

Le Projet

Nous allons donc décomposer le projet en cinq phases qui sont les suivantes

Étape 1 : Cloner le référentiel

Étape 2 : Build Maven

Étape 3 : Révision du code

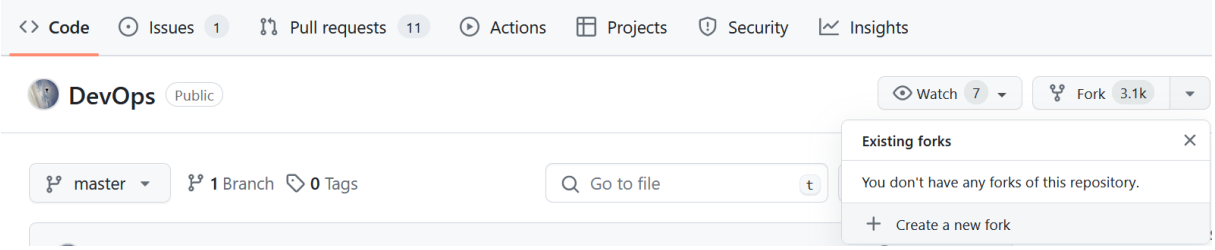
Étape 4 : Télécharger des artefacts

Étape 5 : Déployer l'application

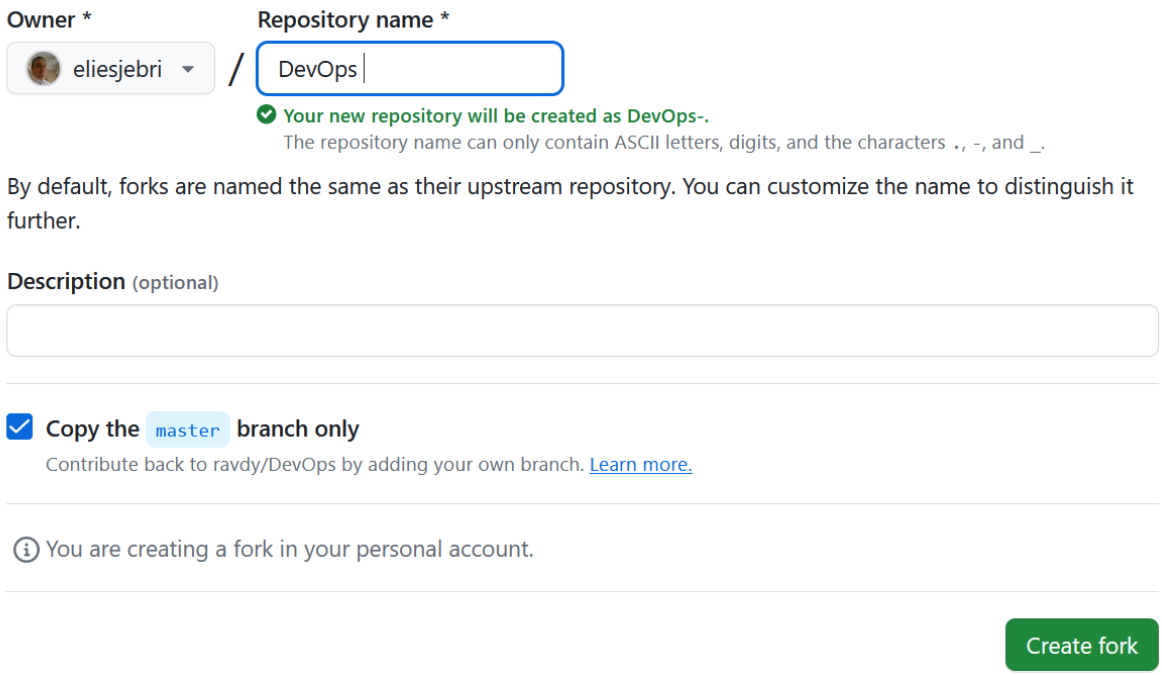
I. Première étape : cloner le référentiel

Donc, dans la première étape de ce pipeline, nous allons cloner le référentiel git https://github.com/eliesjebri/maven-web-app.git depuis votre compte Github.

Depuis votre compte Github allez à « Create a new fork »

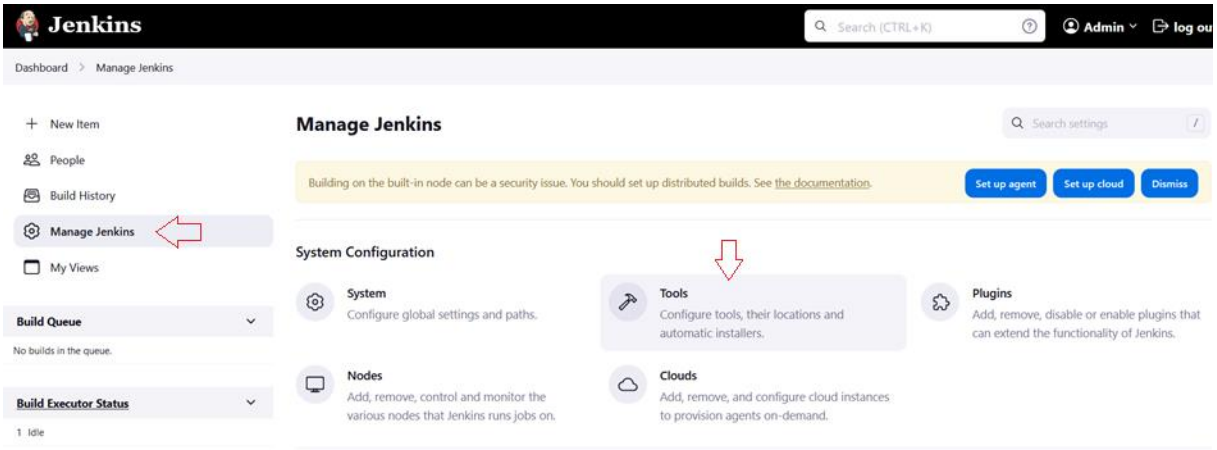


Puis créez le fork

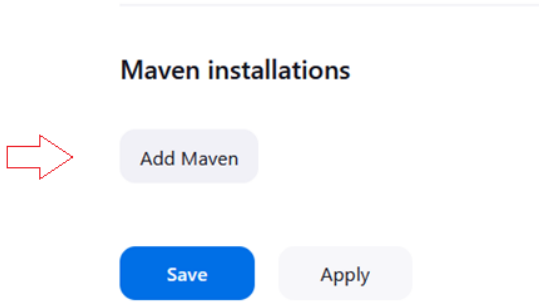


a) Commençons maintenant à configurer Jenkins :

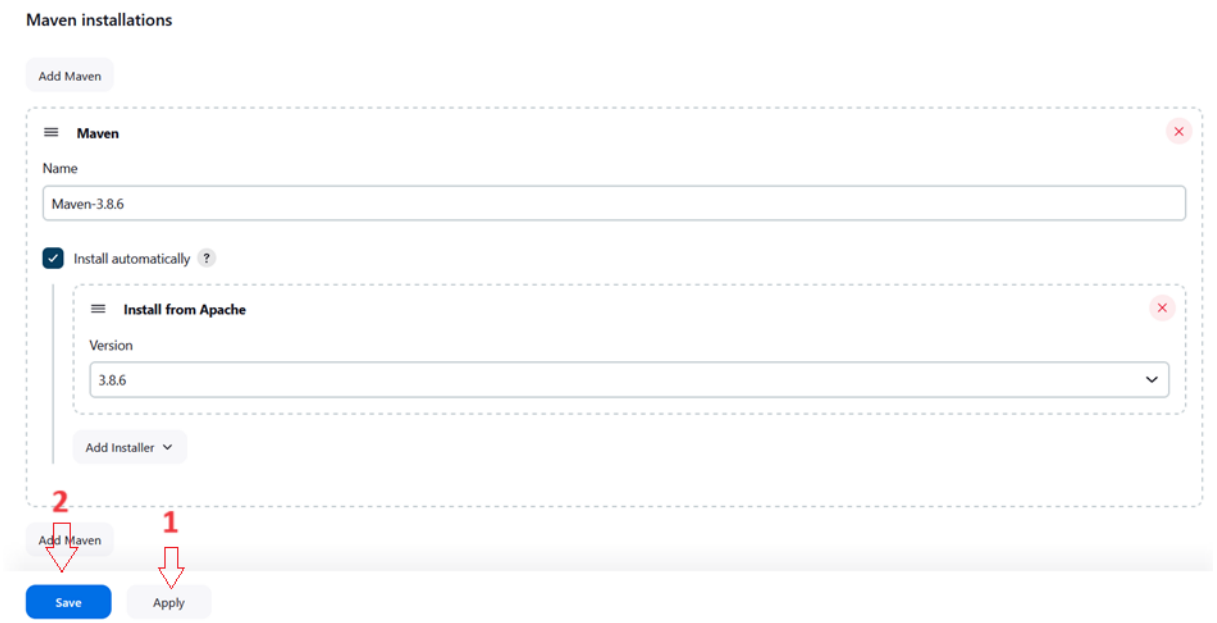
Tout d'abord, nous devons configurer maven en tant que configuration globale d'outil dans le tableau de bord Jenkins.



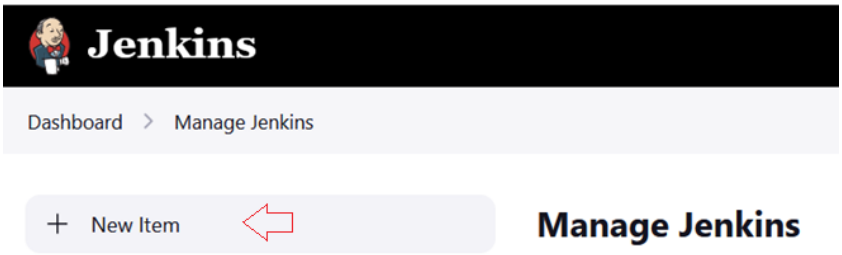
-



-

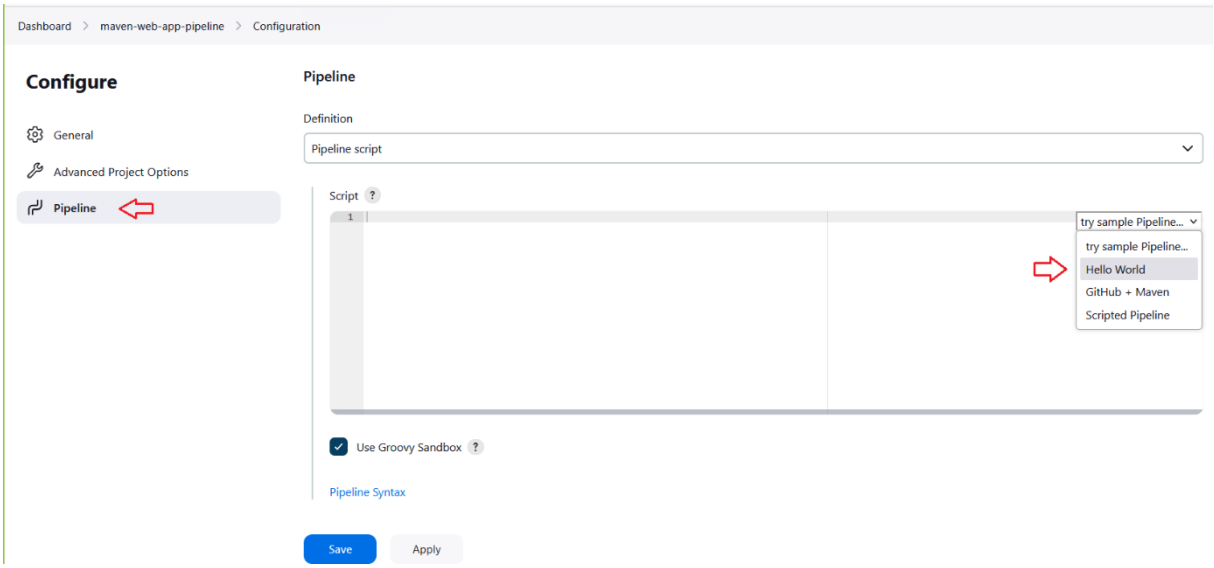


b) Commençons maintenant à construire le pipeline :

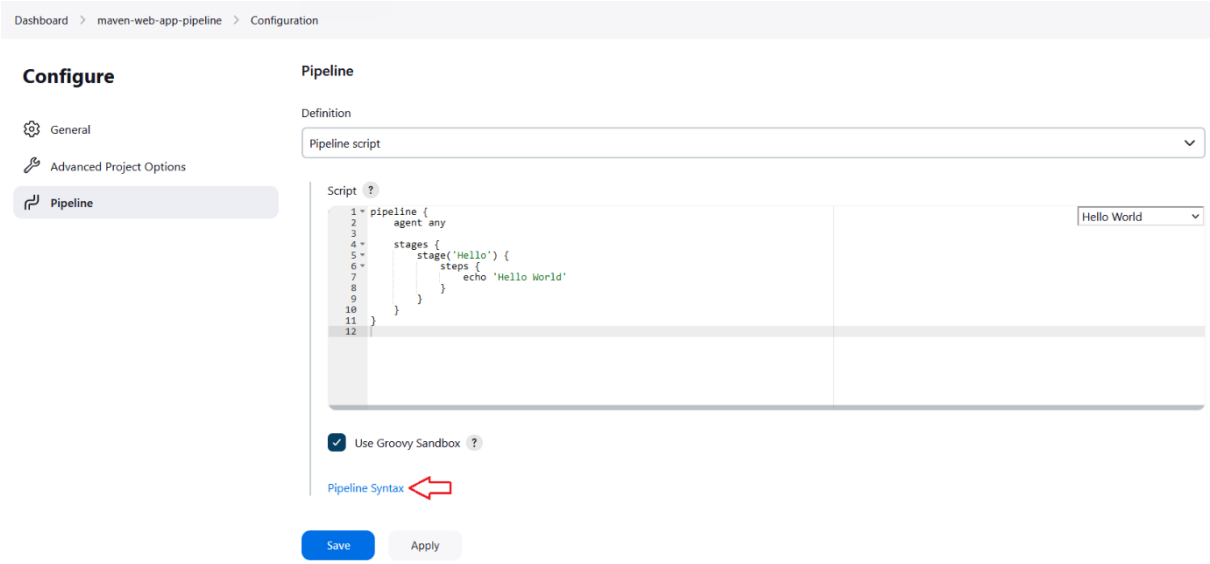


-

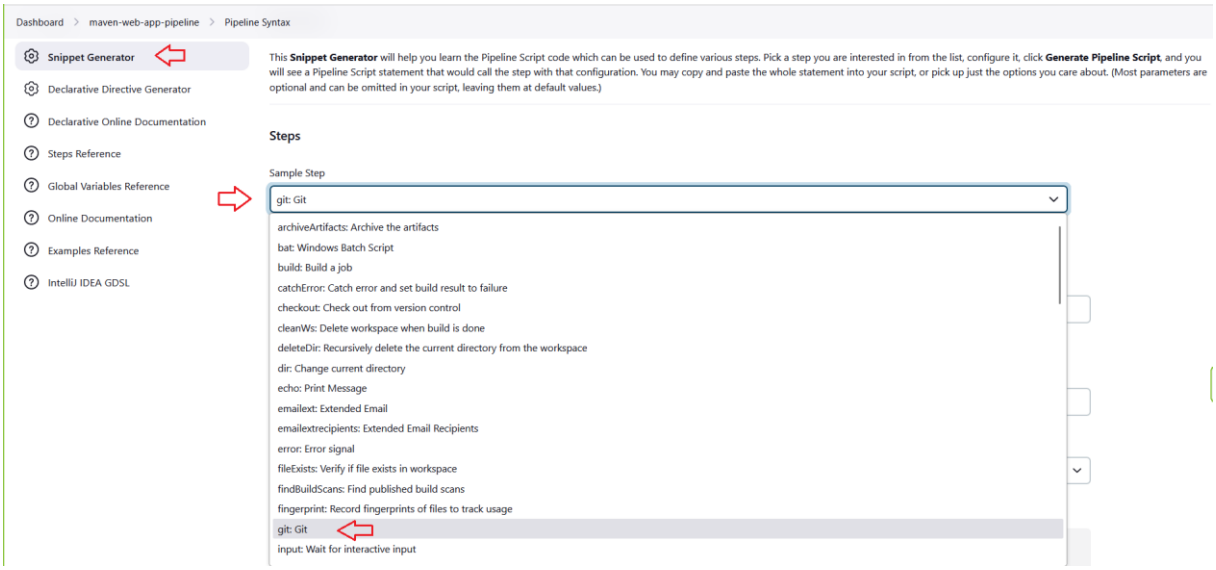
-



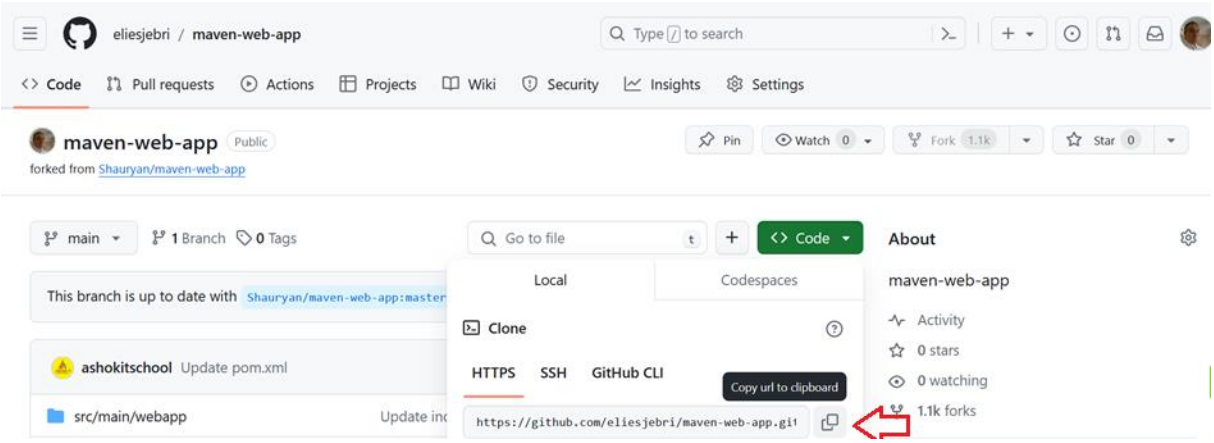
-


Dashboard > maven-web-app-pipeline > Configuration
Configure
General
Advanced Project Options
Pipeline
Pipeline
Definition
Pipeline script
Script
pipeline {
agent any
stages {
stage('Hello') {
steps {
echo 'Hello World'
}
}
}
}
Hello World
Use Groovy Sandbox
Pipeline Syntax
Save
Apply

-


Dashboard > maven-web-app-pipeline > Pipeline Syntax
Snippet Generator
Declarative Directive Generator
Declarative Online Documentation
Steps Reference
Global Variables Reference
Online Documentation
Examples Reference
IntelliJ IDEA GDSL
This Snippet Generator will help you learn the Pipeline Script code which can be used to define various steps. Pick a step you are interested in from the list, configure it, click Generate Pipeline Script, and you will see a Pipeline Script statement that would call the step with that configuration. You may copy and paste the whole statement into your script, or pick up just the options you care about. (Most parameters are optional and can be omitted in your script, leaving them at default values.)
Steps
Sample Step
git: Git
archiveArtifacts: Archive the artifacts
bat: Windows Batch Script
build: Build a job
catchError: Catch error and set build result to failure
checkout: Check out from version control
cleanWs: Delete workspace when build is done
deleteDir: Recursively delete the current directory from the workspace
dir: Change current directory
echo: Print Message
emailext: Extended Email
emailextrecipients: Extended Email Recipients
error: Error signal
fileExists: Verify if file exists in workspace
findBuildScans: Find published build scans
fingerprint: Record fingerprints of files to track usage
git: Git
input: Wait for interactive input

-


eliesjebri / maven-web-app
Type / to search
Code
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
maven-web-app
Public
forked from Shauryan/maven-web-app
Pin
Watch 0
Fork 1.1k
Star 0
main
1 Branch
0 Tags
Go to file
Code
About
This branch is up to date with Shauryan/maven-web-app:master
Local
Codespaces
maven-web-app
Clone
Activity
0 stars
0 watching
1.1k forks
ashokitschool Update pom.xml
HTTPS
SSH
GitHub CLI
Copy url to clipboard
src/main/webapp
https://github.com/eliesjebri/maven-web-app.gi

-

Steps

Sample Step

git: Git

git

Repository URL

https://github.com/eliesjebri/maven-web-app.git

Failed to connect to repository : Error performing git command: ls-remote -h https://github.com/eliesjebri/maven-web-app.git HEAD

Branch

main

Credentials

- none -

+ Add

-

Include in polling?

Poll remote repository for changes. Default is 'true'.

If poll is false, then the remote repository will not be polled for changes. If poll is true or is not set, then the remote repository will be polled for changes.

(from Git plugin)

Include in changelog?

Compute changelog for this job. Default is 'true'.

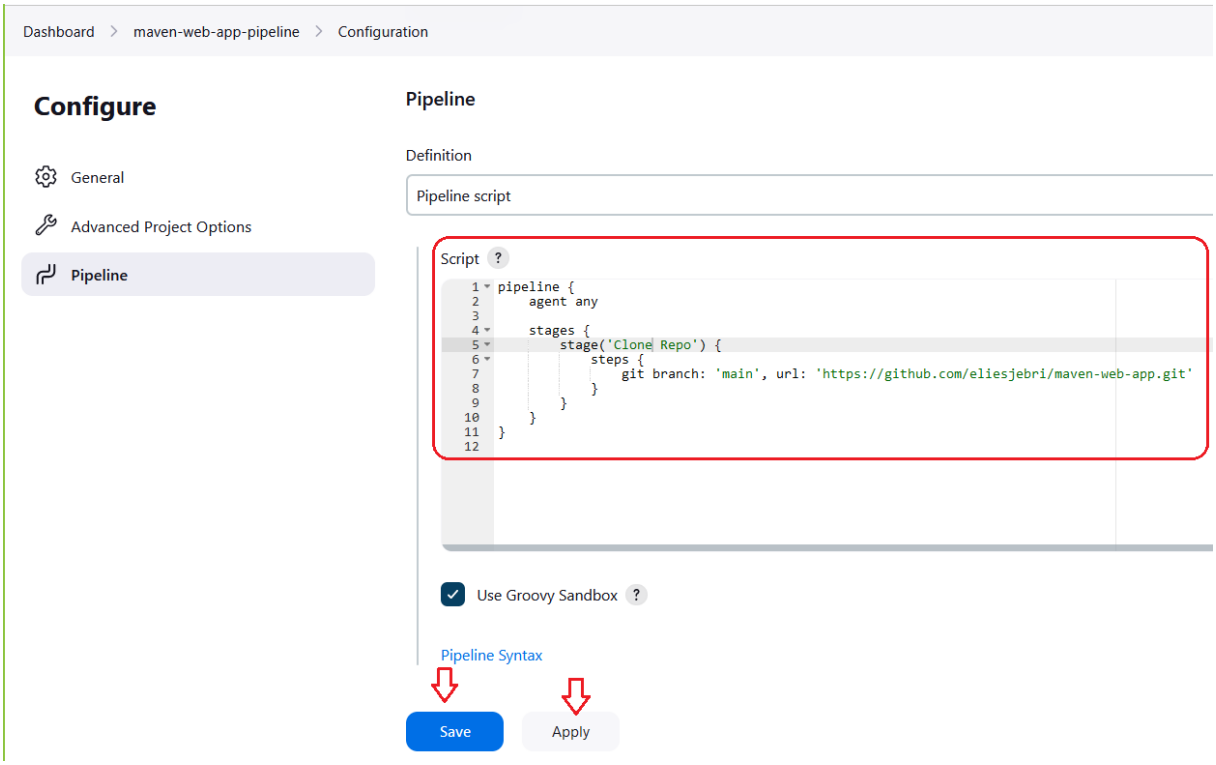
If changelog is false, then the changelog will not be computed for this job. If changelog is true or is not set, then the changelog will be computed.

(from Git plugin)

Generate Pipeline Script

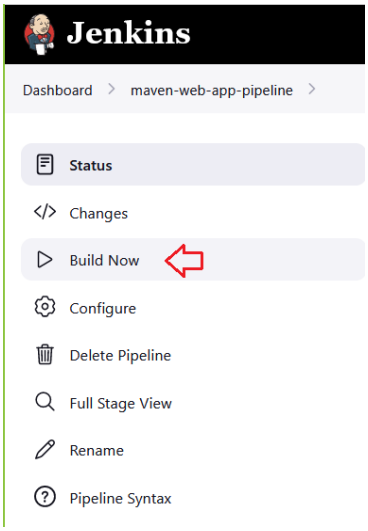
```
git branch: 'main', url: 'https://github.com/eliesjebri/maven-web-app.git'
```

-

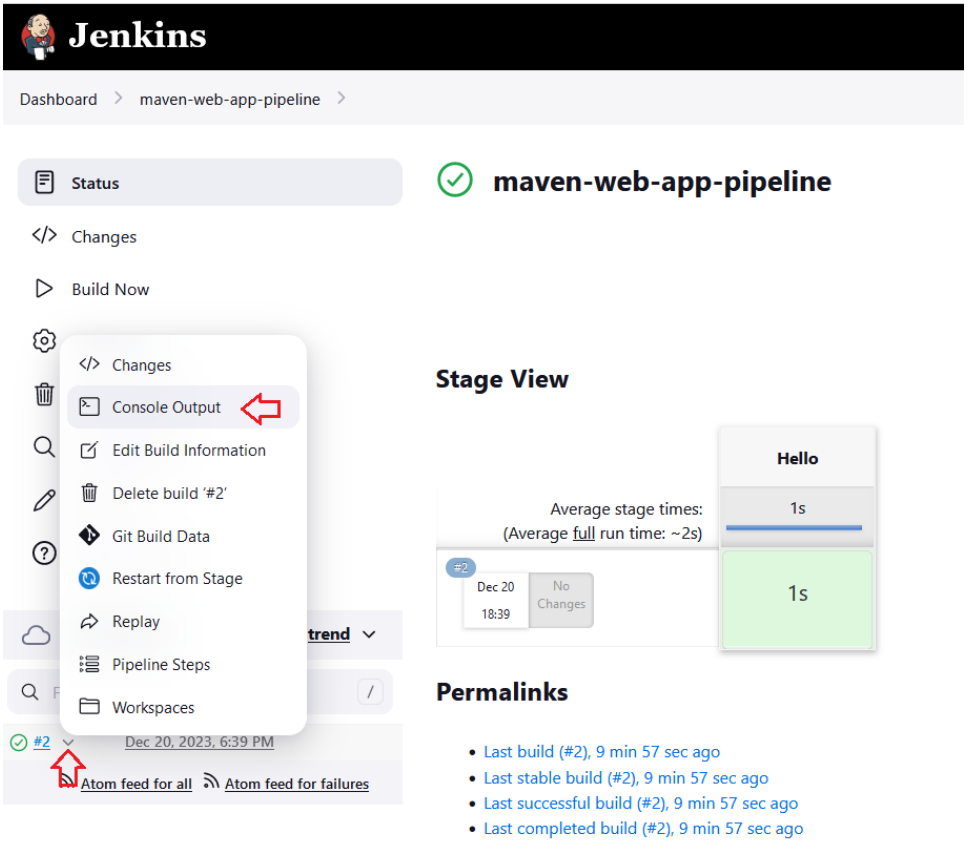


Maintenant, Appliquez et enregistrez.

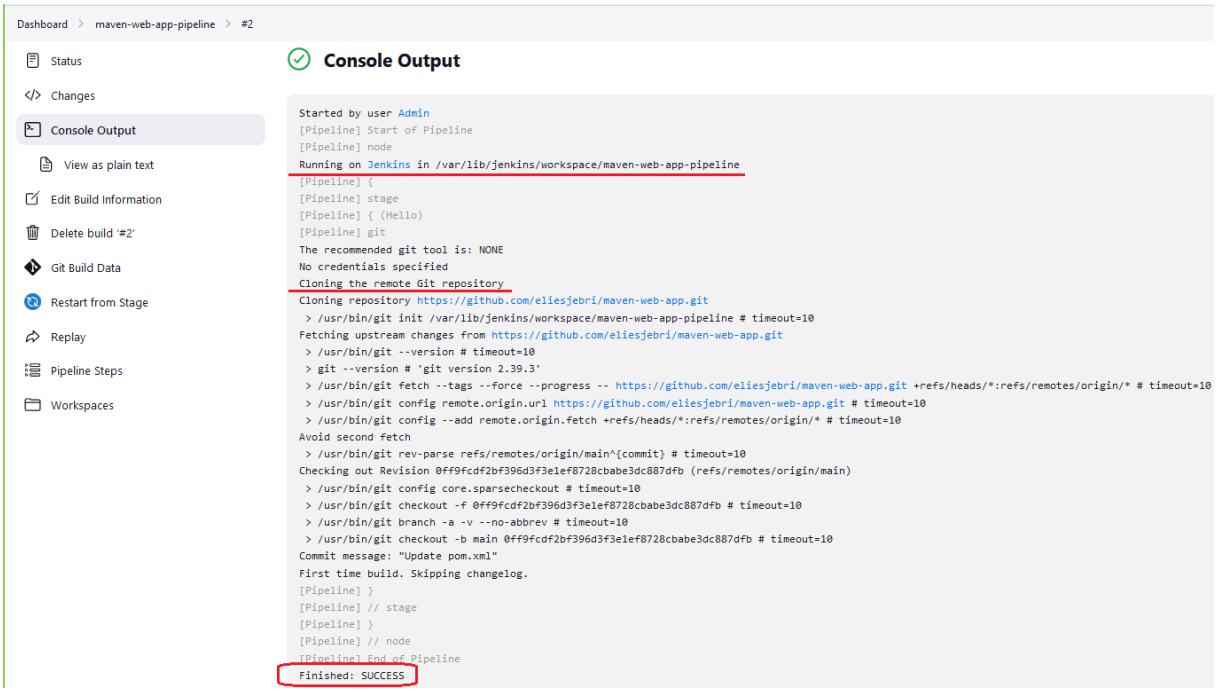
c) Exécutez simplement le pipeline et voyez si la première étape est en cours de traitement ou non. Alors cliquez sur Construire maintenant :



-



-



Nous avons donc terminé notre première étape. Passons maintenant à la deuxième étape qui est Maven Build.

II. Deuxième étape : construction de Maven

Pipeline

Definition

Pipeline script

Script ?

```
pipeline {
    agent any

    stages {
        stage('Clone Repo') {
            steps {
                git branch: 'main', url: 'https://github.com/eliesjebri/maven-web-app.git'
            }
        }
        stage('Build Stage') {
            steps {
                sh "mvn clean package"
            }

            tools {
                maven 'Maven-3.8.6'
            }
        }
    }
}
```

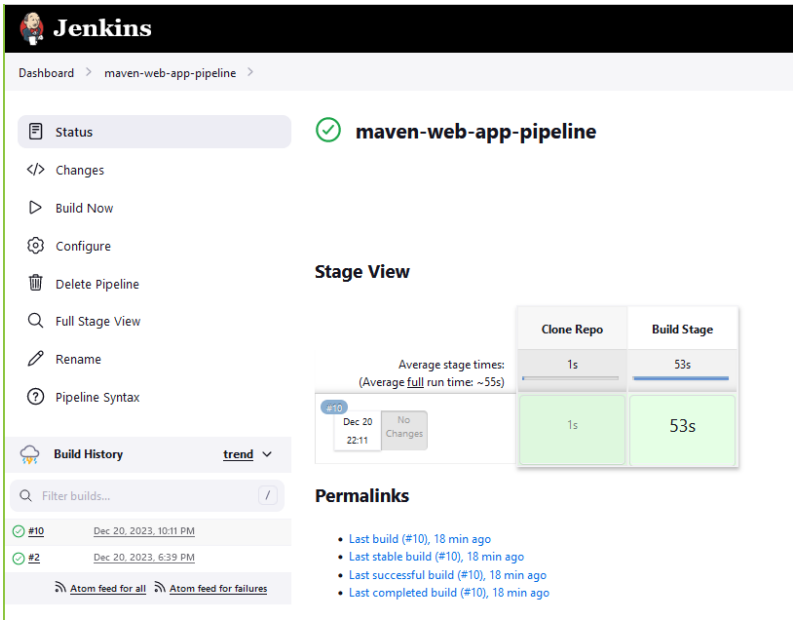
Use Groovy Sandbox ?

Pipeline Syntax

Save Apply

Maintenant, appliquez et enregistrez.

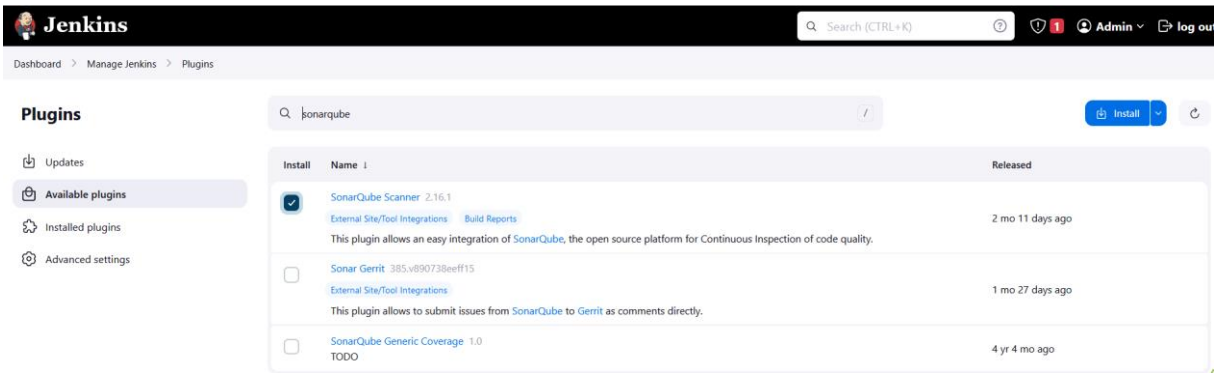
Exécutez simplement le pipeline et voyez si la deuxième étape est en cours de traitement ou non. Alors cliquez sur **Construire maintenant.**



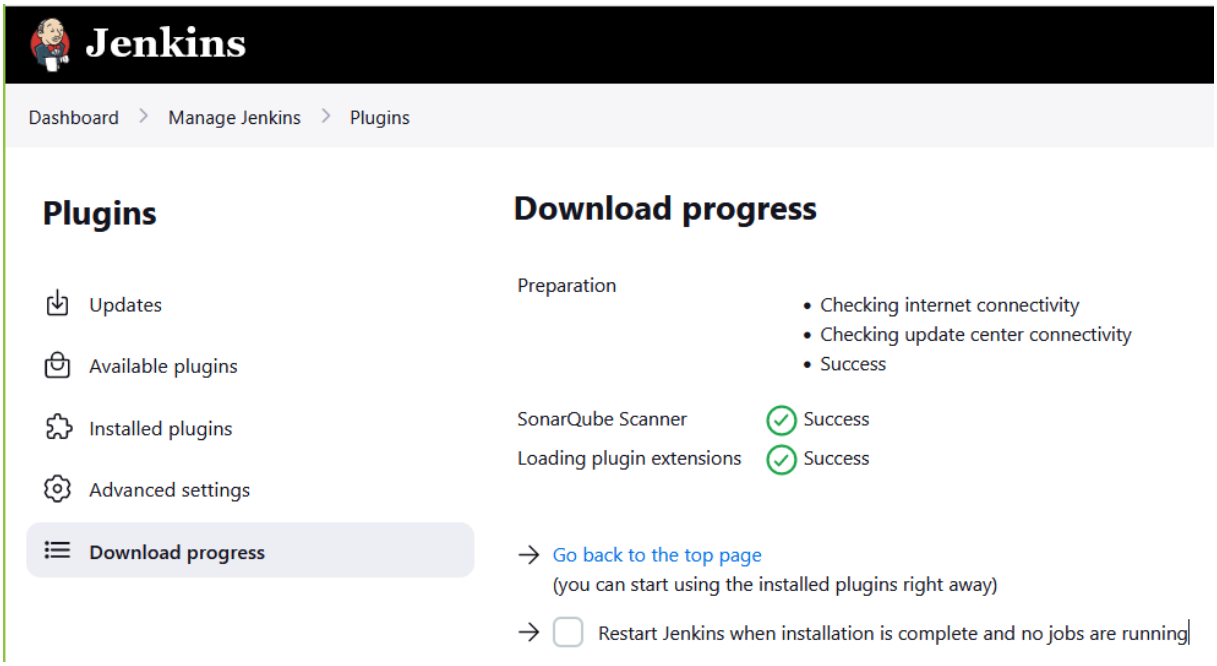
Ainsi, alors que la deuxième étape est terminée, à savoir le build avec Maven, nous allons maintenant passer à la troisième étape, à savoir la révision du code.

III. Étape 3 : révision du code

Cette étape sera entièrement prise en charge par Sonarqube pour la revue et l'analyse du code. Pour cela, nous devons intégrer Sonarqube avec Jenkins Server. Afin d'intégrer Sonarcube à Jenkins, nous devons installer un plugin nommé SonarQube Scanner. Sélectionnez ce plugin et installez-le sans redémarrer.

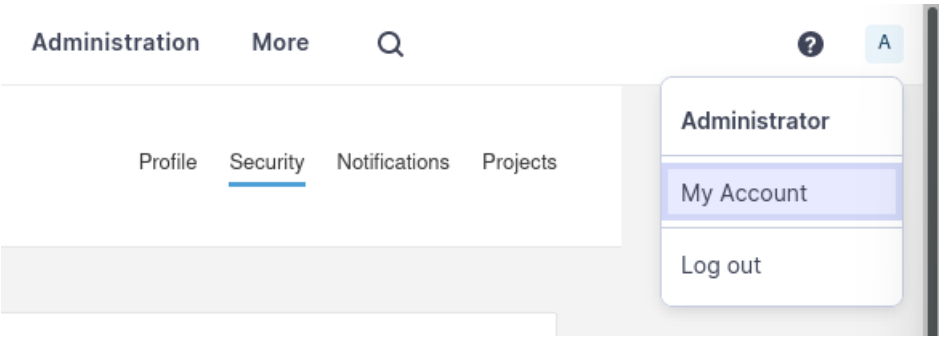


-

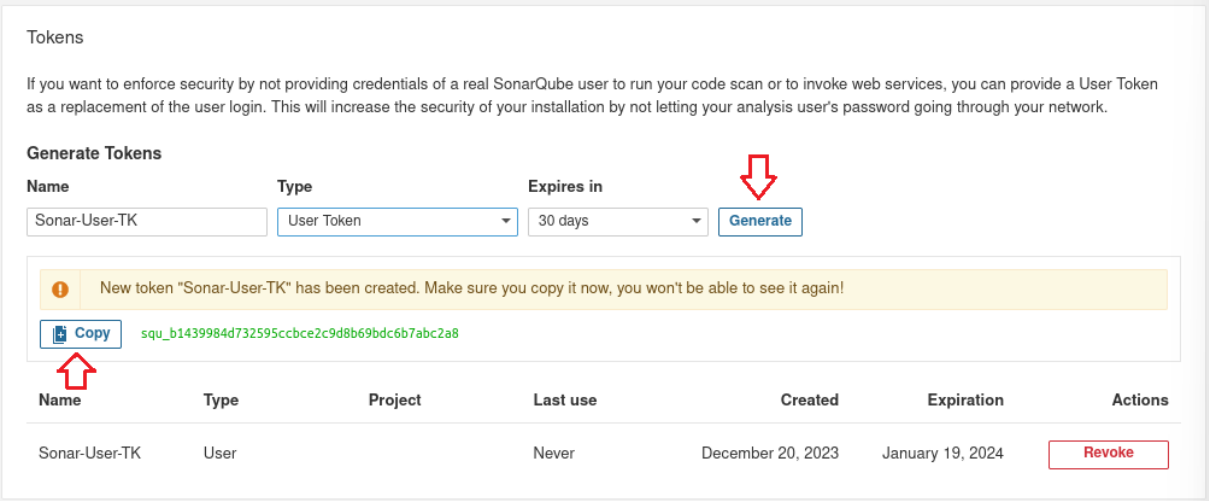


Connectez-vous maintenant à SonarQube, puis accédez à Adminstrator >> Compte >> Sécurité > > Générer un jeton

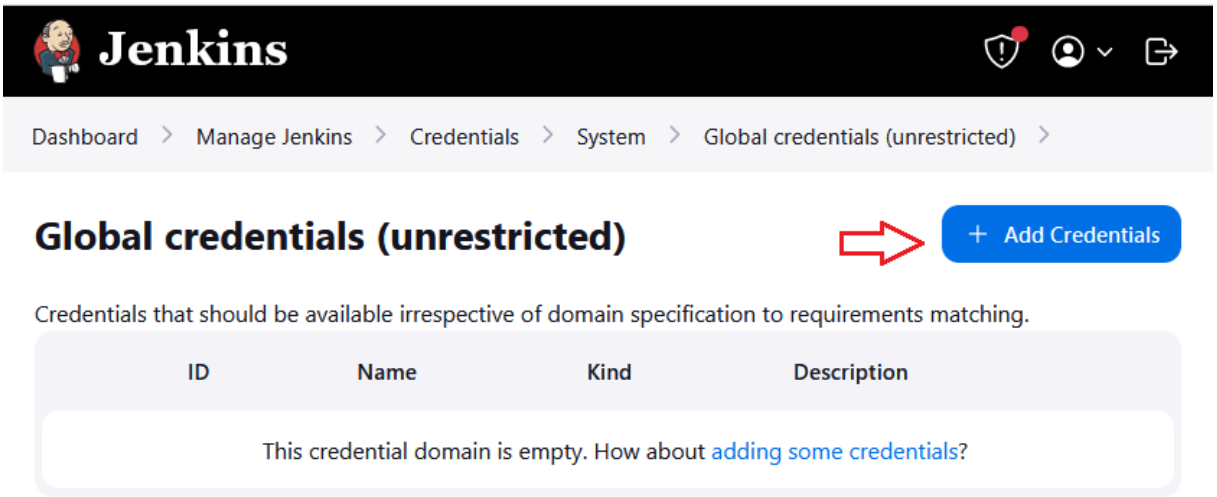
Ici, générez un jeton utilisateur et copiez le jeton dans le panneau Jenkins.



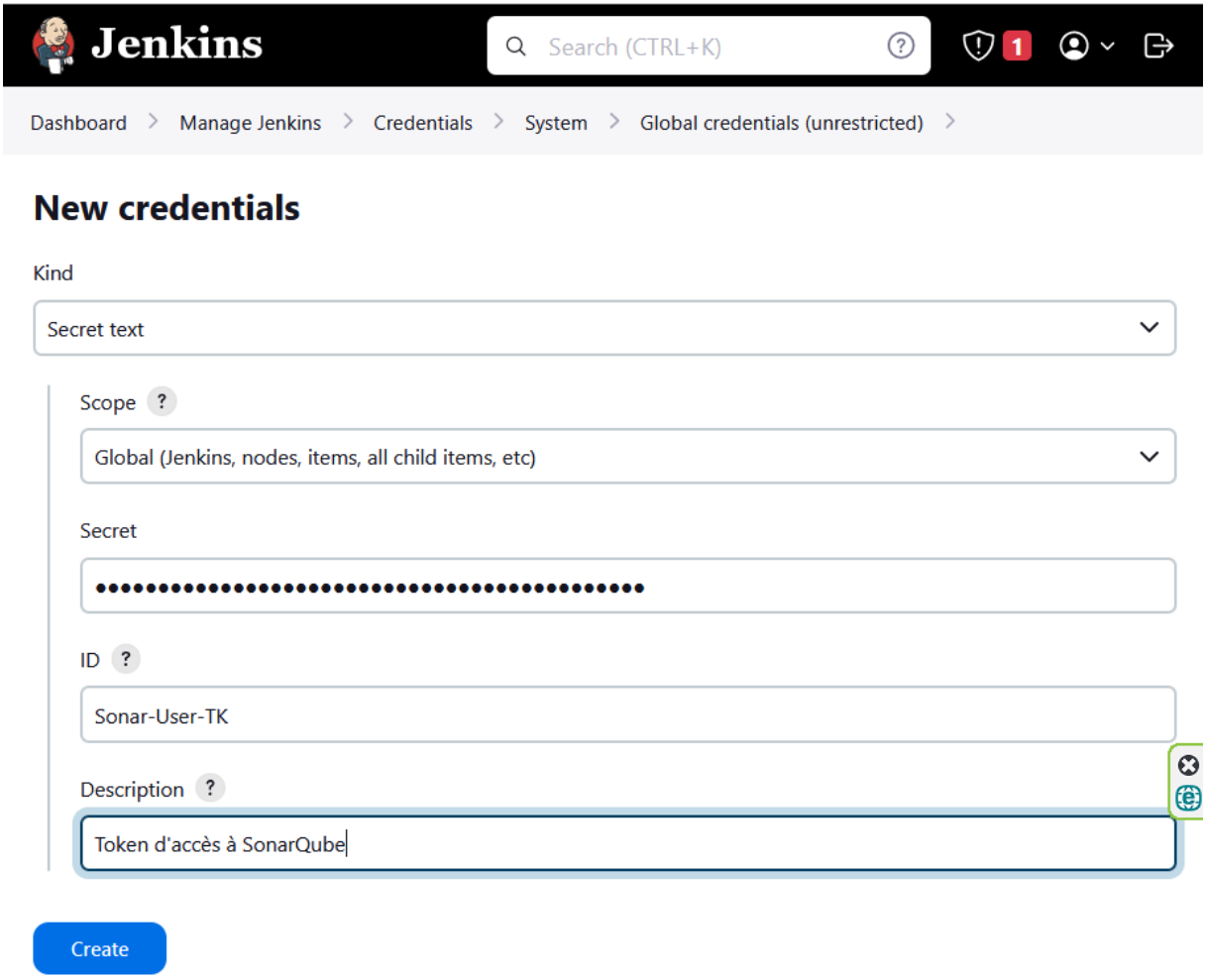
-



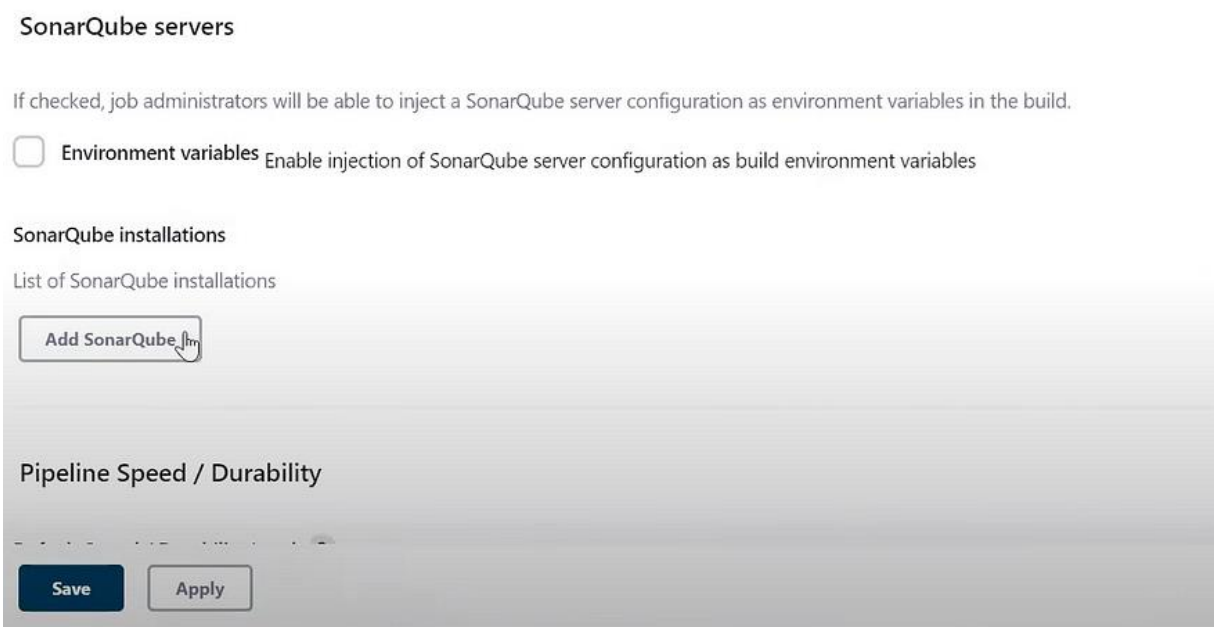
Ajoutez vos nouvelles informations d'identification de SonarQube dans Jenkins en vous déplaçant vers le chemin Dashboard - Manage Jenkins Credentials - System - Global credentials (unrestricted). Ajoutez votre clé secrète ici.



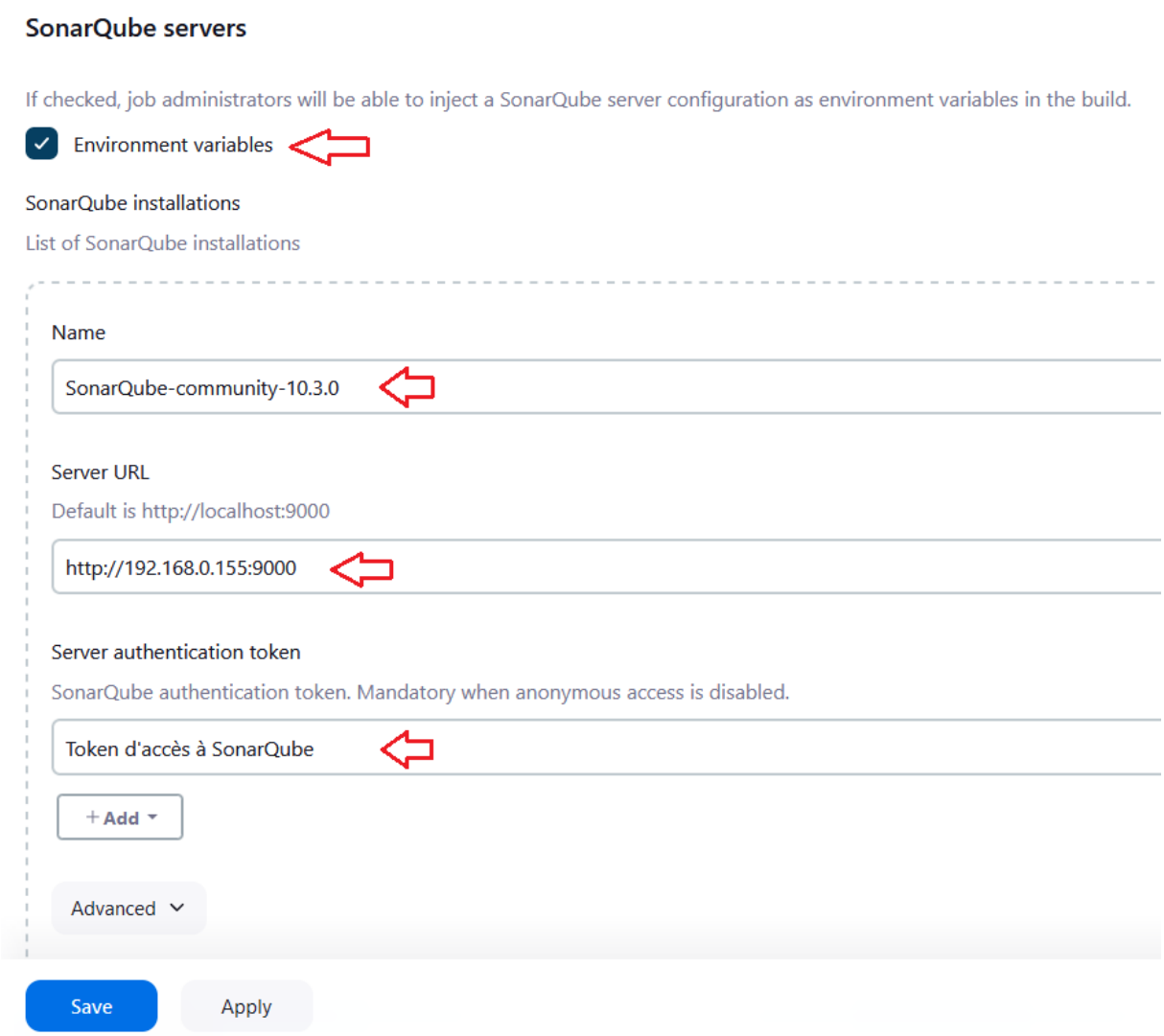
-



Dans le tableau de bord Jenkins, accédez à Dashboard - Manage Jenkins - System – SonarQube servers.



-



Cliquez sur Appliquer et Enregistrer. Avec cela, vous avez configuré Jenkins avec SonarQube. Passons maintenant au pipeline et ajoutons une étape pour l'étape SonarQube.

Pipeline

Definition

Pipeline script

Script ?

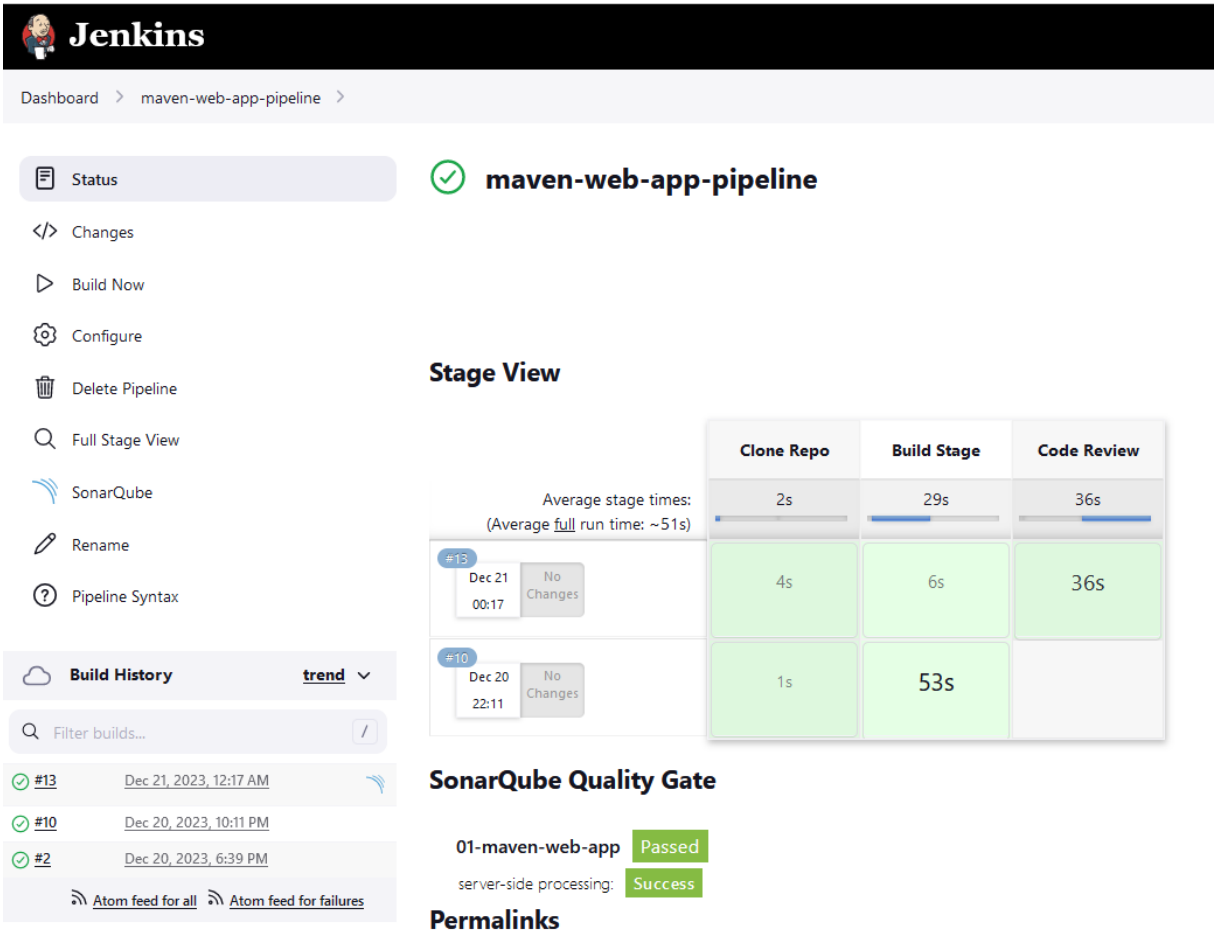
```
pipeline {
    agent any

    stages {
        stage('Clone Repo') {
            steps {
                git branch: 'main', url: 'https://github.com/eliesjebri/maven-web-app.git'
            }
        }
        stage('Build Stage') {
            steps {
                sh "mvn clean package"
            }

            tools {
                maven 'Maven-3.8.6'
            }
        }
        stage('Code Review') {
            steps {
                withSonarQubeEnv('SonarQube-community-10.3.0') {
                    sh "mvn clean package sonar:sonar"
                    echo 'Static Analysis Completed'
                }
            }

            tools {
                maven 'Maven-3.8.6'
            }
        }
    }
}
```

☑ Use Groovy Sandbox ?

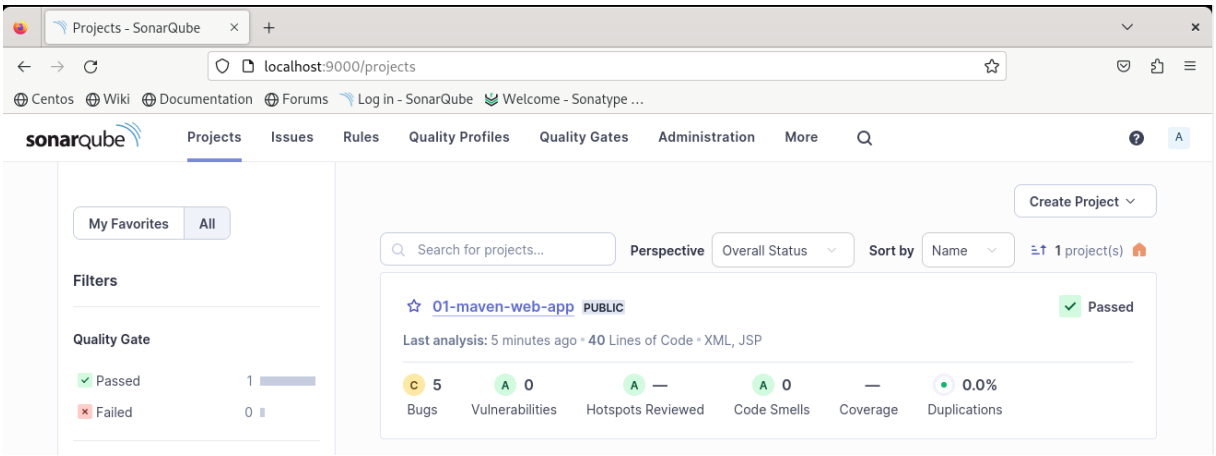
Save Apply

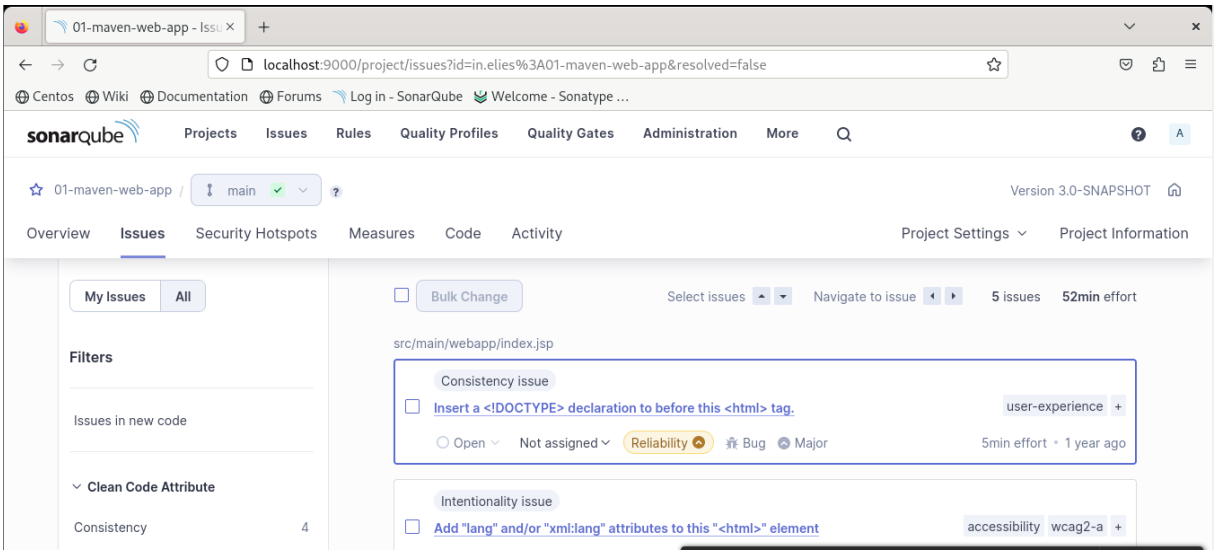
Cliquez sur Appliquer et enregistrez-le pour d'autres procédures.

Maintenant, exécutez Build Now et voyez si le pipeline est entièrement fonctionnel ou non. C’est le moment où nous avons trois étapes en cours.



Si la construction est en phase de réussite, vous constaterez que votre application Web Maven se trouve dans SonarQube.





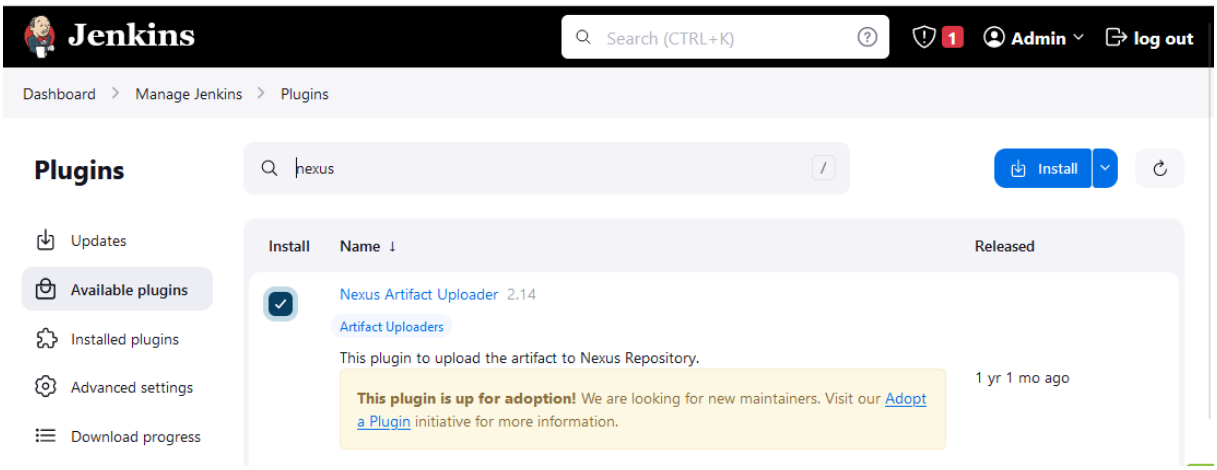
Construisons maintenant notre quatrième étape.

IV. Étape 4 : Télécharger les artefacts

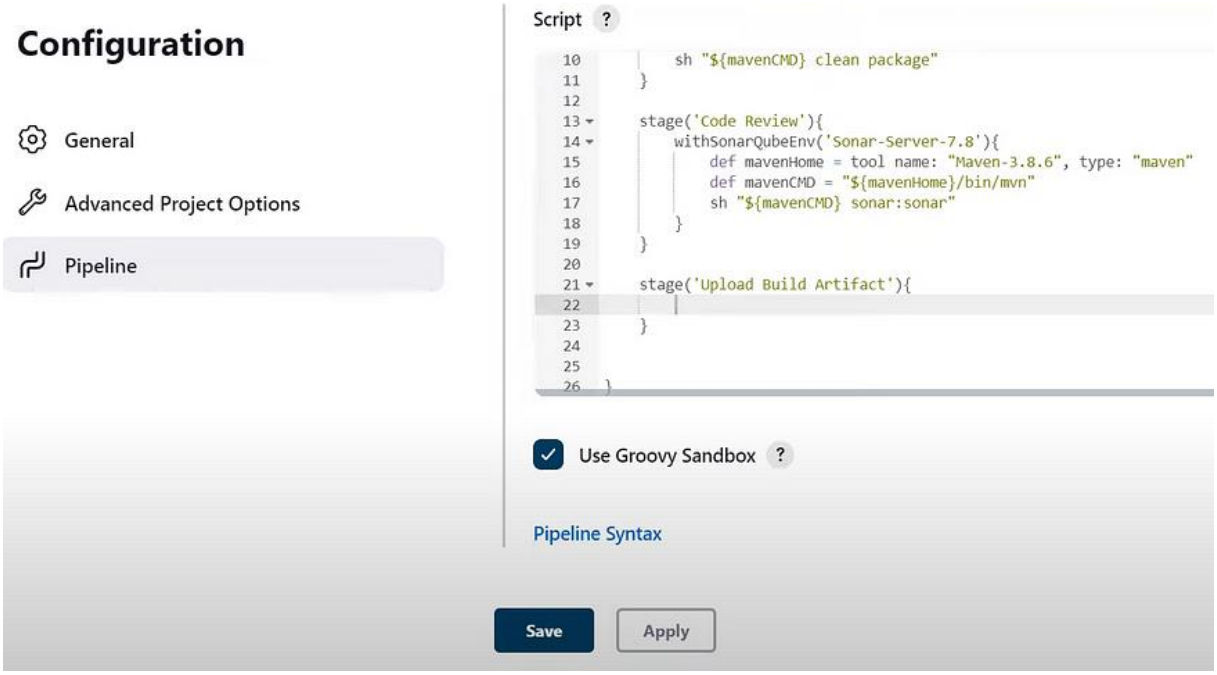
Pour cette étape, nous devons configurer le système de référentiel d'artefacts Nexus dans Jenkins.

Accédez donc au tableau de bord Jenkins Dashboard - Manage Jenkins - Plugins

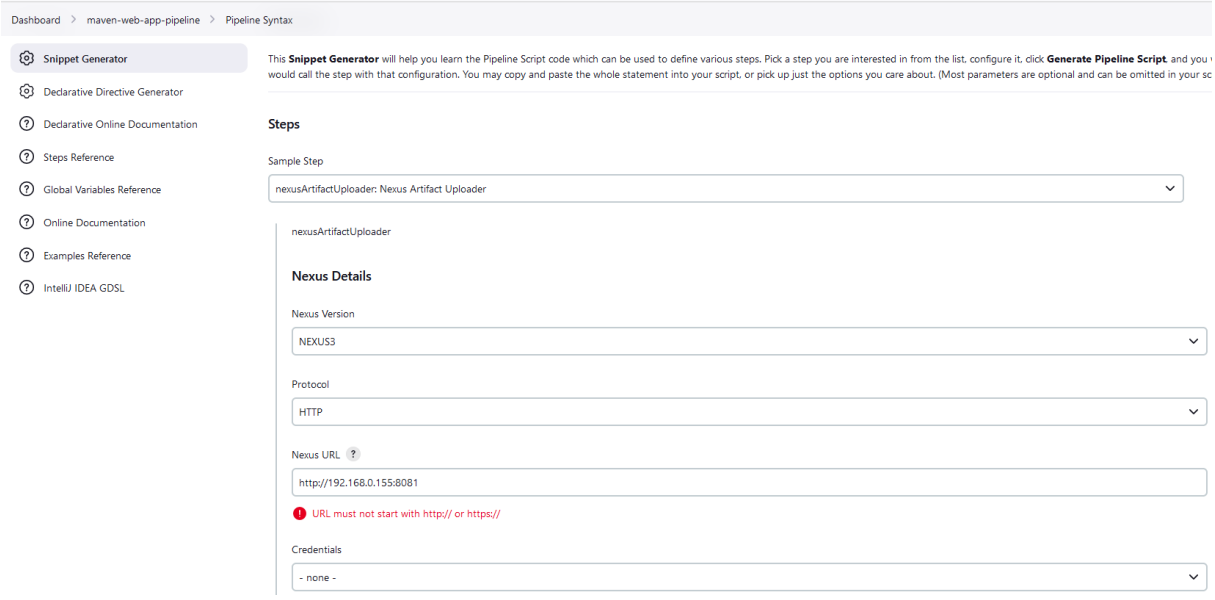
Installez Nexus Artifact Uploader



Ajoutons la quatrième étape dans le Pipeline en vous aidant avec l'assistant.

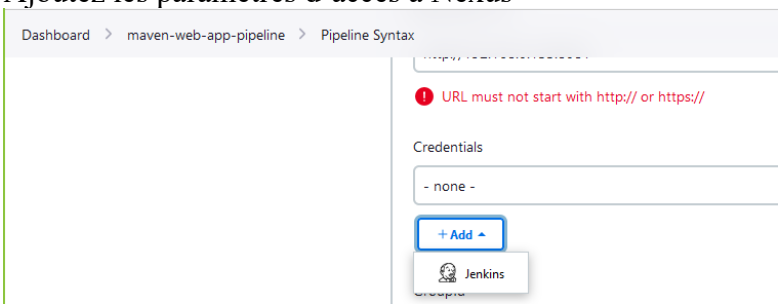


Utilisez le générateur de syntaxe du pipeline pour générer l'étape du script.

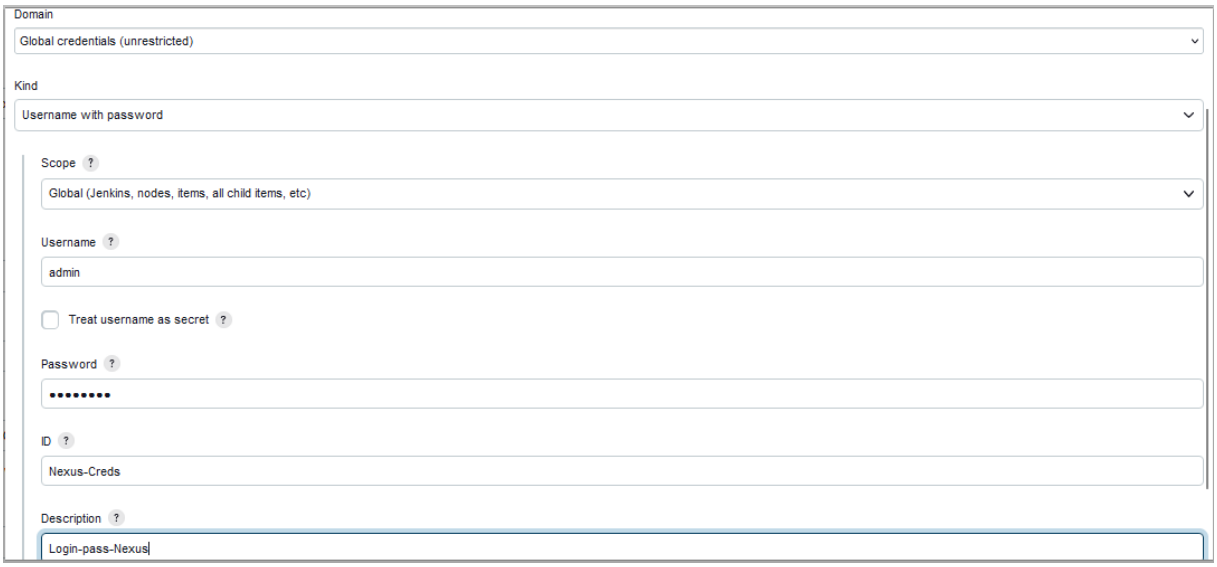


-

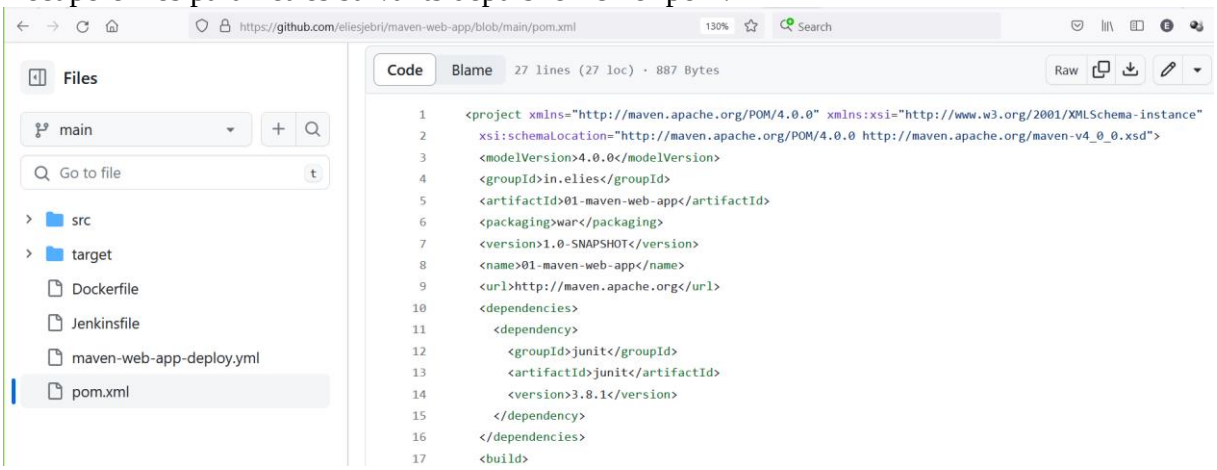
Ajoutez les paramètres d'accès à Nexus



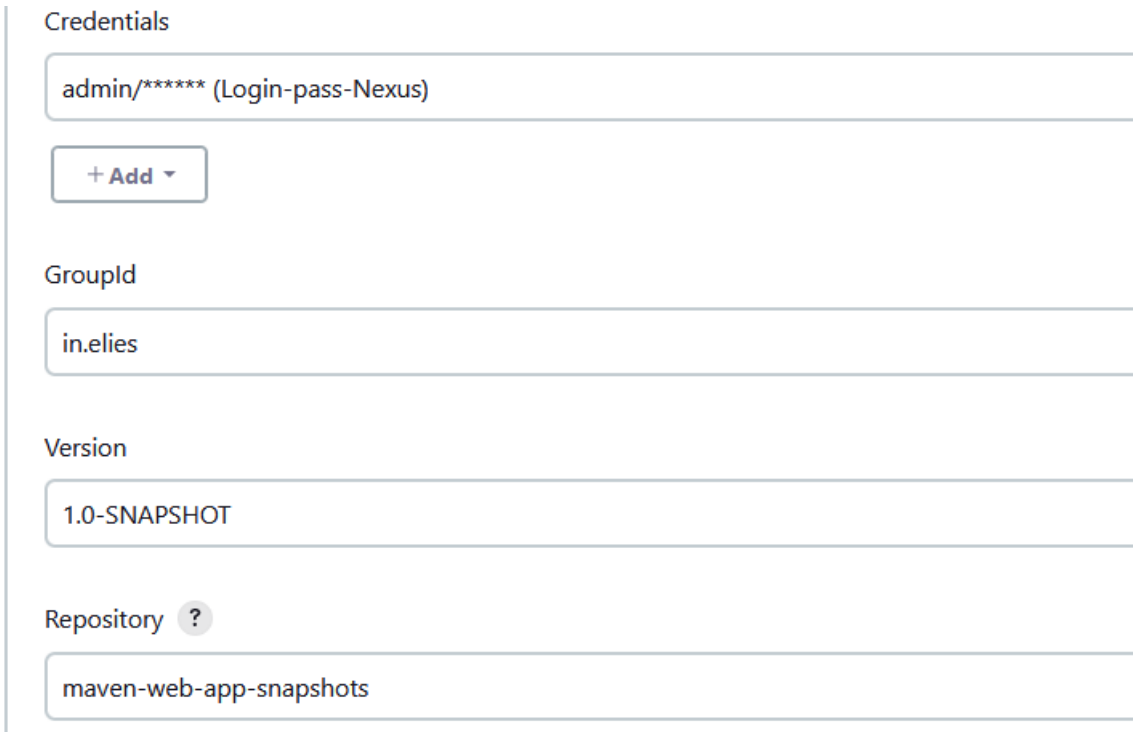
-



Récupérez les paramètres suivants depuis le fichier pom.xml

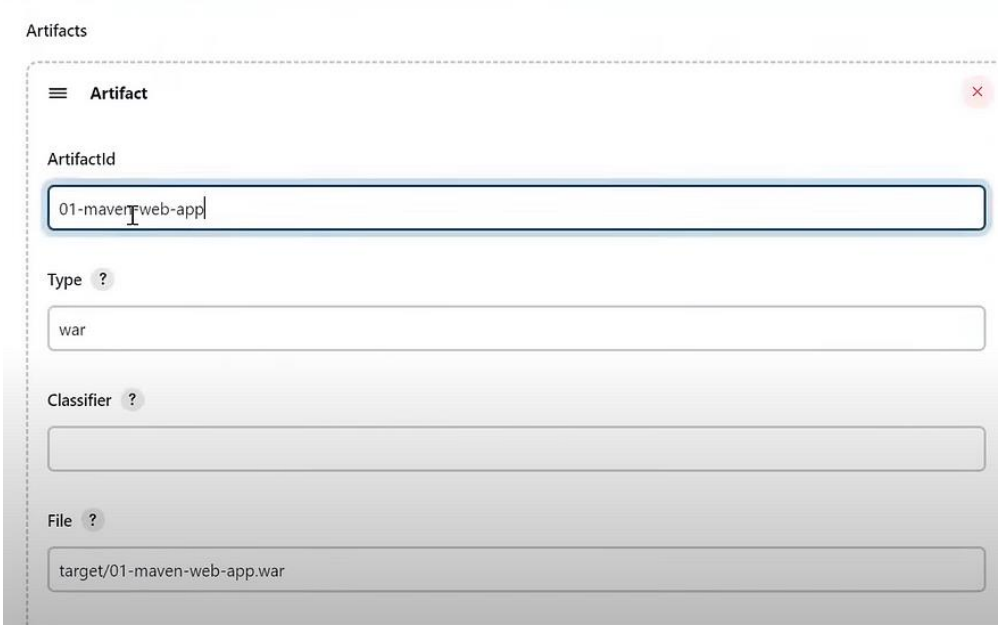


-



Fournissez les détails du référentiel Nexus selon votre configuration dans la syntaxe du pipeline Jenkins.

Ajoutez maintenant les détails aux onglets Artefact.

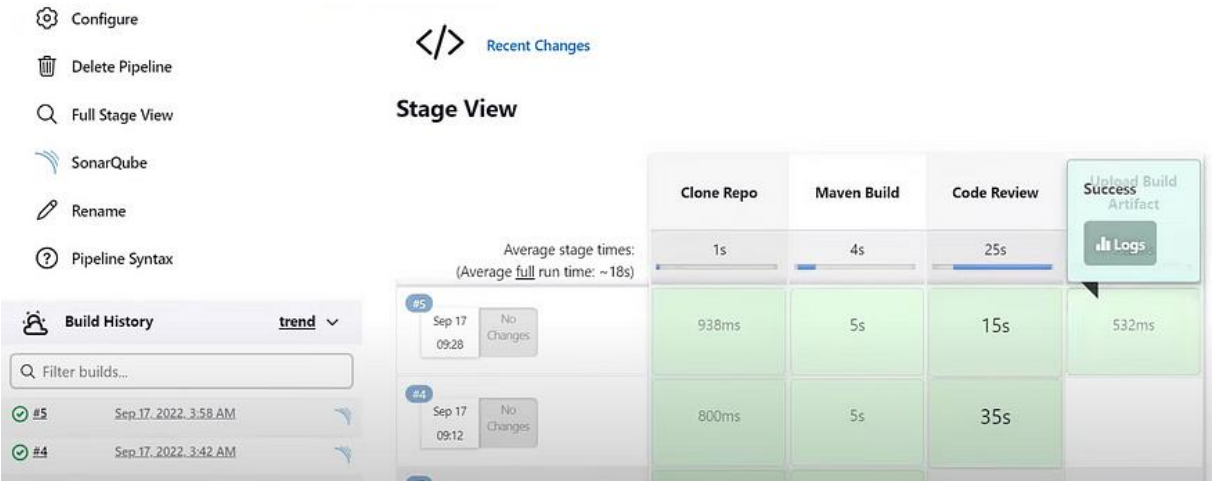


-

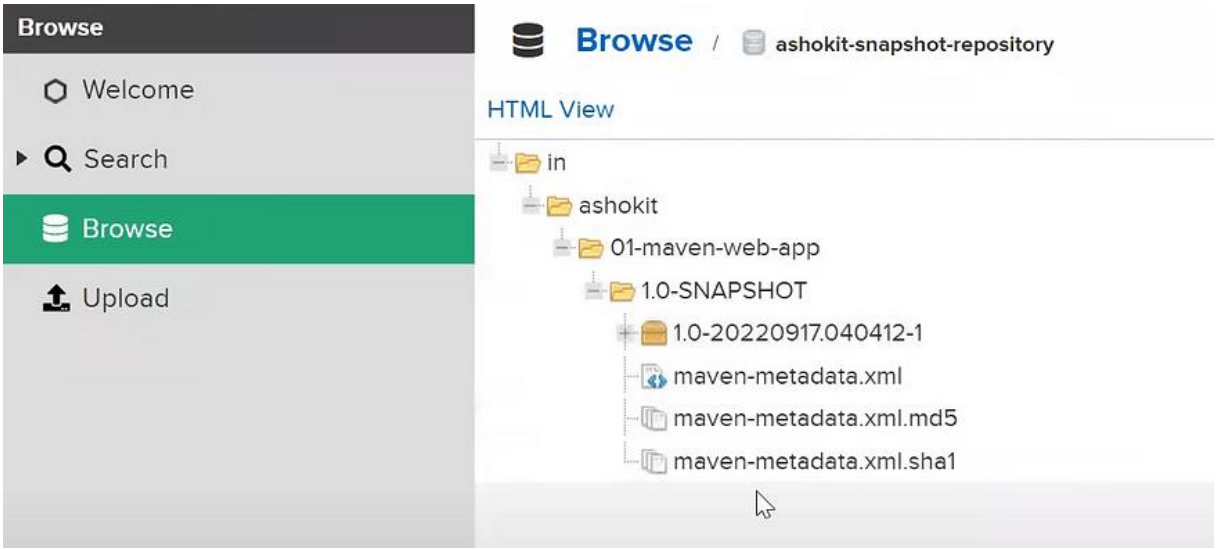
Passez maintenant le script dans l'étape Nexus.

Cliquez sur Appliquer et Enregistrer.

Maintenant, exécutons le build maintenant et vérifions si le pipeline **Télécharger les artefacts** fonctionne ou non. À l'heure actuelle, nous avons 4 étapes différentes à parcourir.



Vérifiez maintenant votre référentiel Nexus, vous trouverez les artefacts téléchargés dans le système d'annuaire.

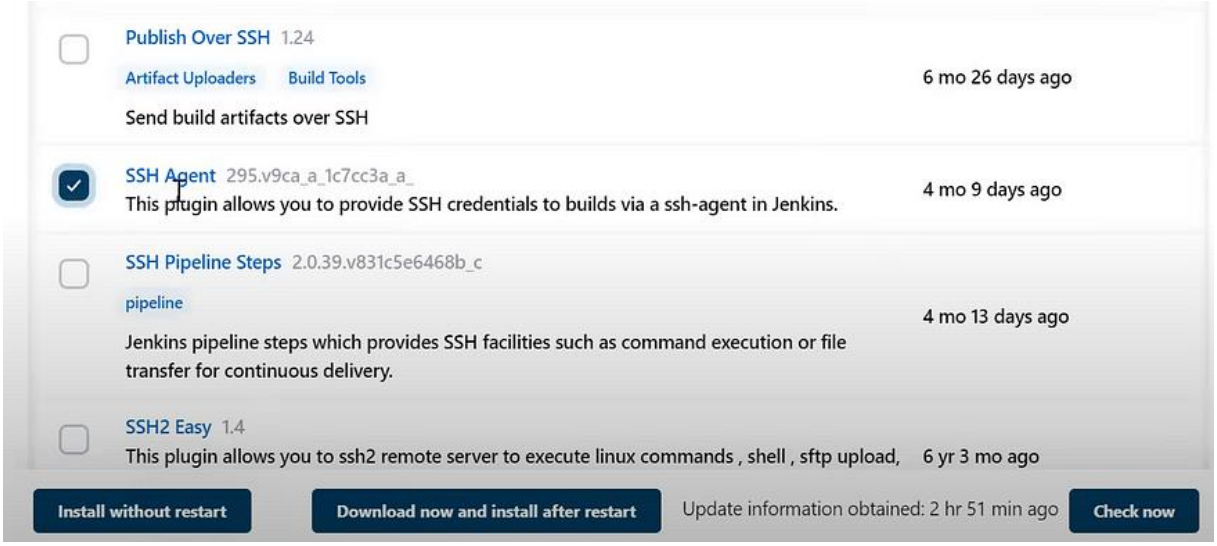


Nous pouvons désormais constater que les 4 étapes ont été franchies avec succès. Nous allons maintenant passer à notre dernière étape de déploiement de l'application.

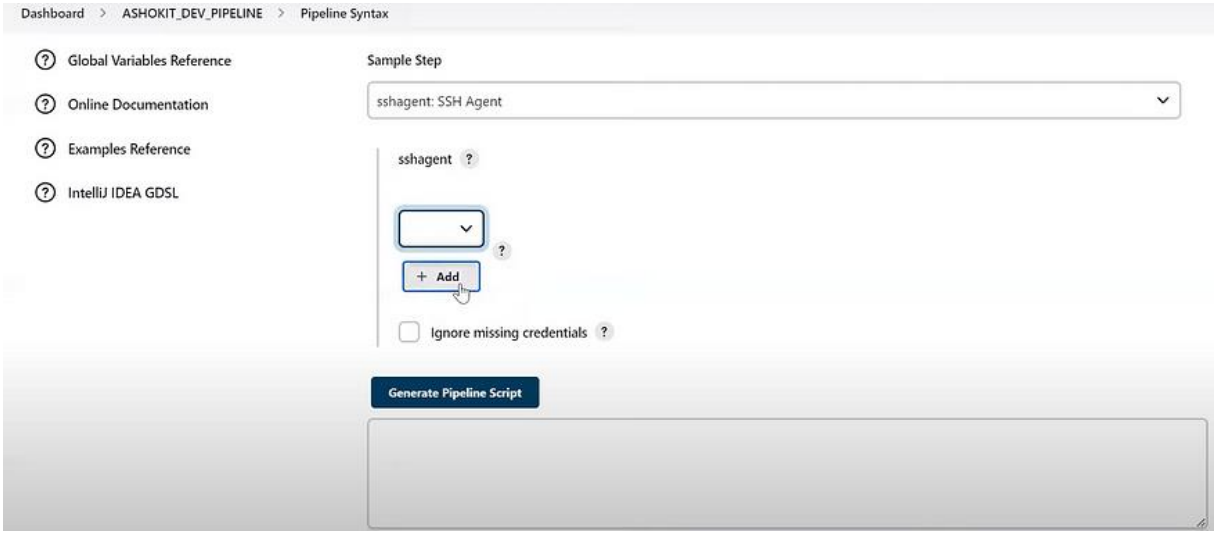
Étape 5 : déployer l'application

Pour le déploiement, nous avons besoin d'un agent ssh. Pour cela, déplacez-vous vers Tableau de bord >> Gérer Jenkins >>Gestionnaire de plugins>> Disponible

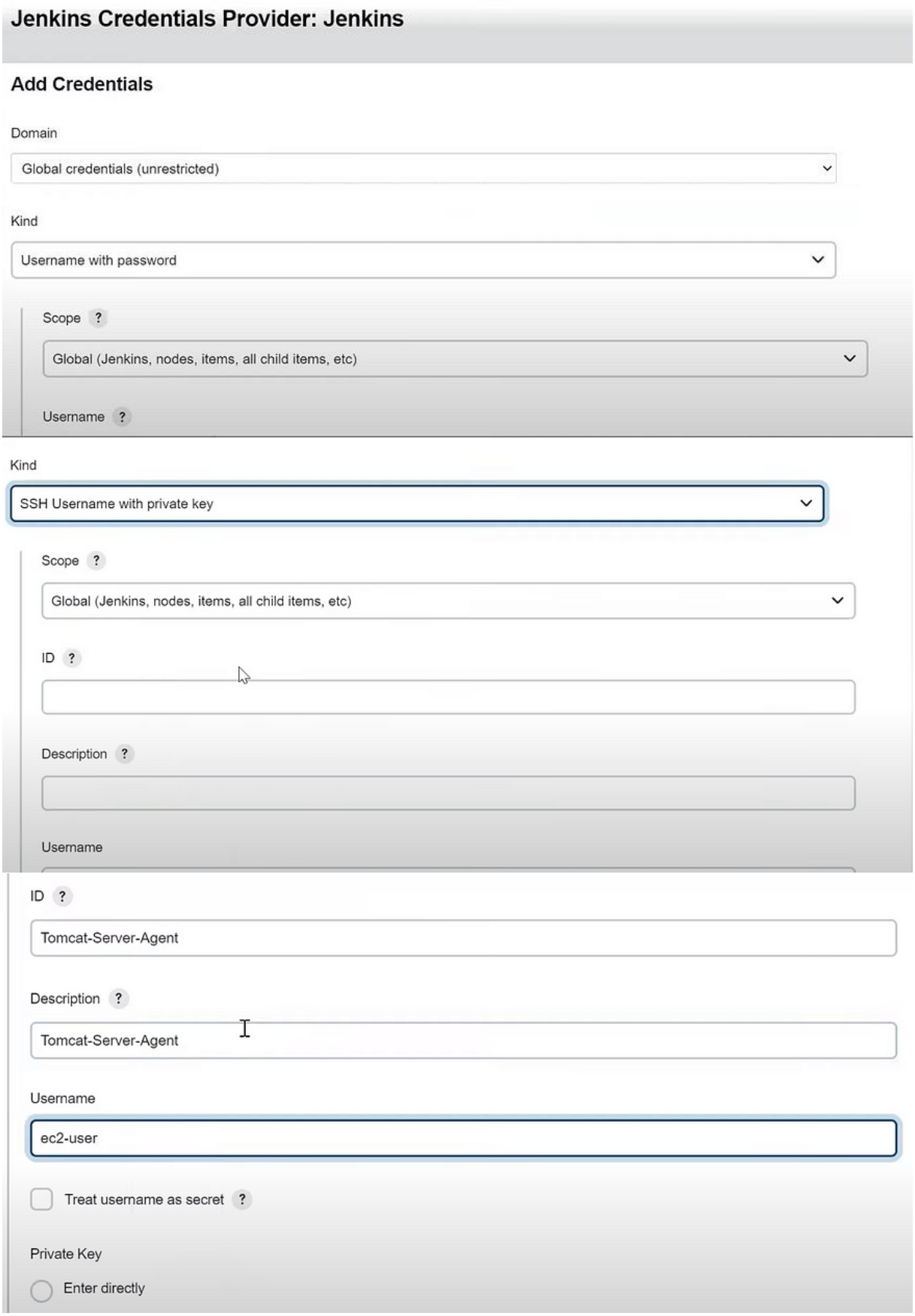
Installez ici, recherchez l'**Agent SSH.** Installez-le sans redémarrer.



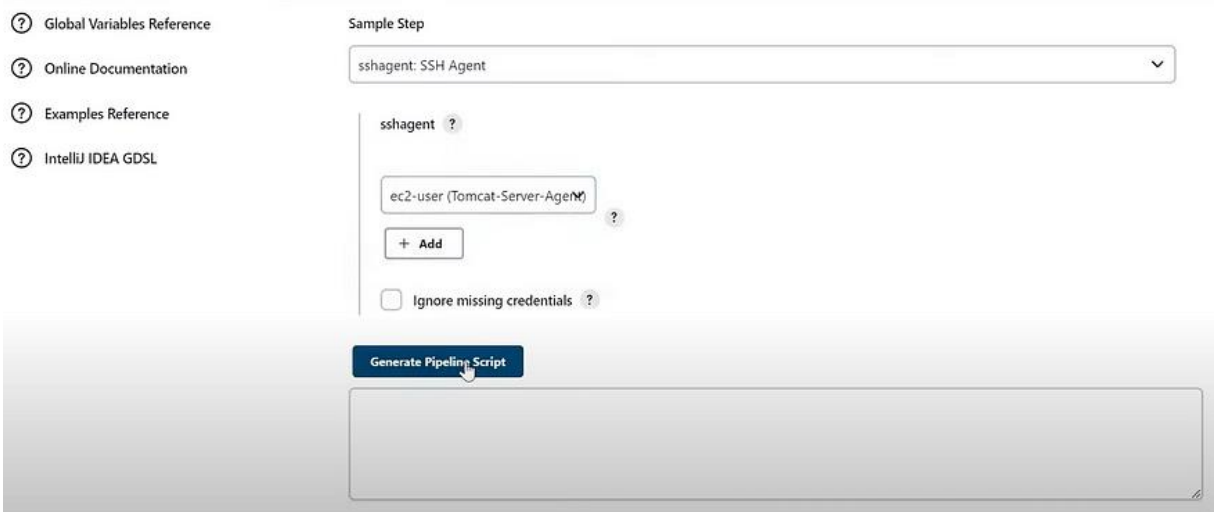
Ajoutons maintenant une étape pour le déploiement. Utilisez la syntaxe du pipeline pour créer le script.



Ajoutez maintenant les informations d'identification pour le serveur Tomcat.


Jenkins Credentials Provider: Jenkins
Add Credentials
Domain
Global credentials (unrestricted)
Kind
Username with password
Scope
Global (Jenkins, nodes, items, all child items, etc)
Username
Kind
SSH Username with private key
Scope
Global (Jenkins, nodes, items, all child items, etc)
ID
Description
Username
ID
Tomcat-Server-Agent
Description
Tomcat-Server-Agent
Username
ec2-user
Treat username as secret
Private Key
Enter directly

Fournissez le fichier de clé privée **.pem** pour le serveur Tomcat et cliquez sur AJOUTER. Générez maintenant la syntaxe du pipeline pour la scène.

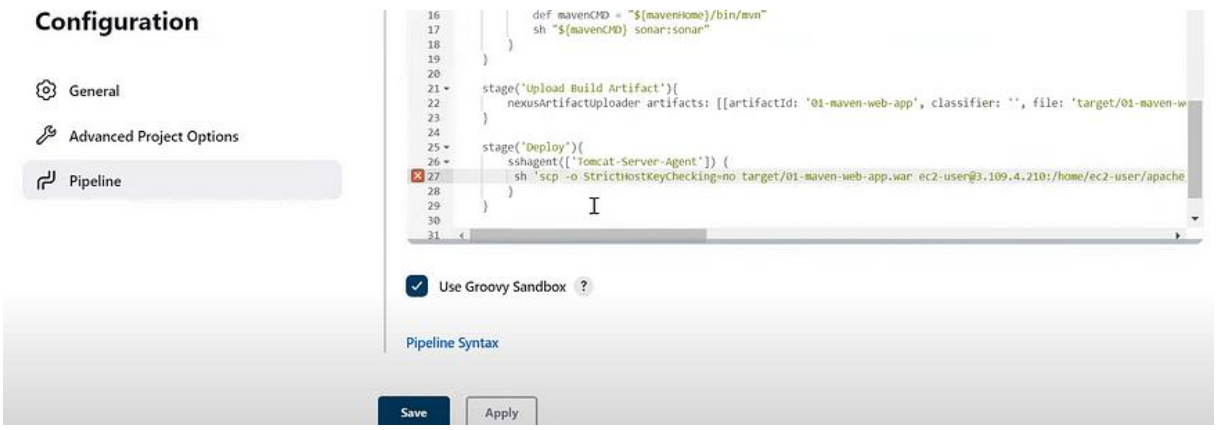


Nous devons maintenant ajouter la cinquième étape au script pour le déploiement de notre application Web.

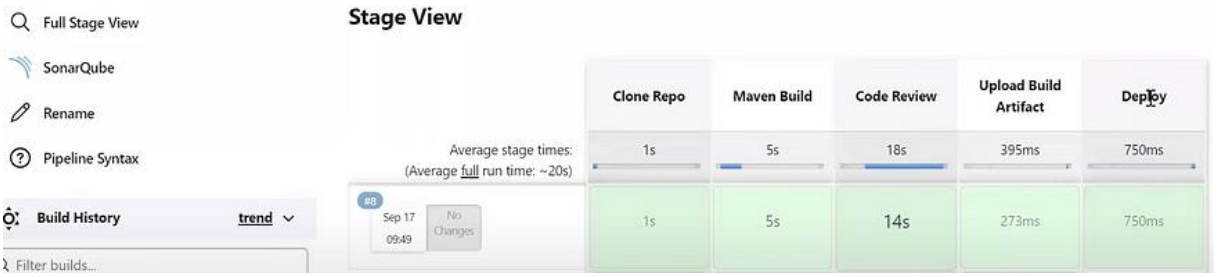
```
sshagent(['Tomcat-Server-Agent']) {
   sh 'scp -o StrictHostKeyChecking=no target/01-maven-web-app.war
   ec2-user@3.109.4.210:/home/ec2-user/apache-tomcat-9.0.65/webapps'
}
```

```
source : target/01-maven-web-app.war

destination: /home/ec2-user/apache-tomcat-9.0.65/webapps
```

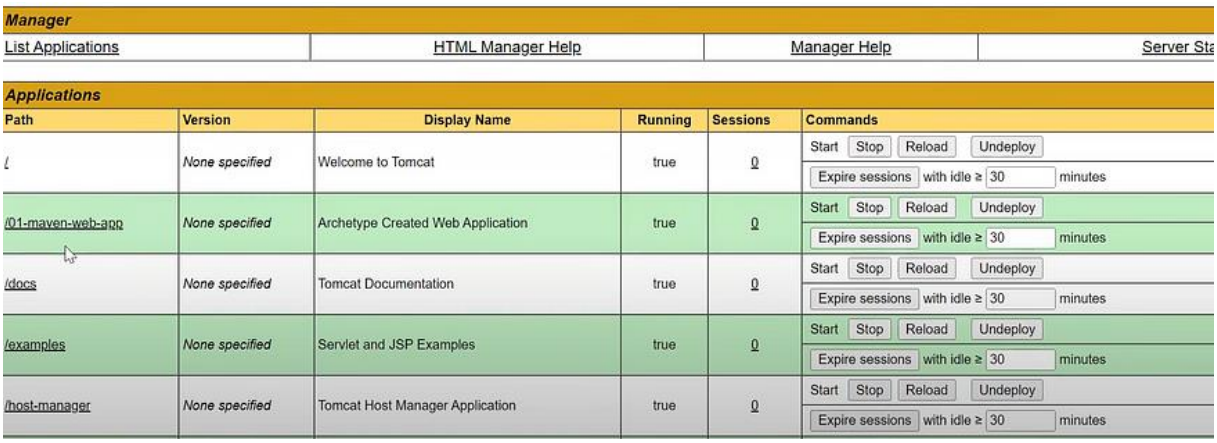
Ajoutez la scène.



Cliquez sur Appliquer et Enregistrer et exécutez la construction maintenant.



Une fois votre build lancé avec succès, vous pouvez maintenant vérifier sur le serveur Tomcat l'application Web en cours d'exécution.



Voici donc le pipeline CICD complet.